

The folder link :

[ML Pipeline Updates](#)

Jun 1, 2026

ML Pipeline Understanding :Autism 2024 and Young vs Older 2024

1. Purpose

This document summarizes my understanding of the ML pipeline steps and the results produced so far for the Autism 2024 and Young vs Older 2024 datasets. The goal is to explain what each phase does, what files/results were produced, and what still needs confirmation before final interpretation.

I worked through the pipeline up to Phase 4.4. Phase 4.4 was started, but SHAP attempted to run on thousands of models, so I asked the team whether SHAP should be limited to the top selected models only.

2. Datasets Used

Study	File used	Note
Autism 2024	2024 FINAL - Autism Study MAR2024 ALL (AM+NM) Features.xlsx 2024 FINAL - Autism Study MAR2024 ALL (AM+NM) Features.xlsx	This is the assigned Autism dataset.
Young vs Older 2024	2024 ALL Post-Processing Speeds Young and Older Adults ALL Features - FINAL AUGUST 20.xlsx 2024 ALL Post-Processing Speeds Young and Older Adults ALL Features - FINAL AUGUST 20.xlsx	Brett confirmed to use only this Post-Processing file and not the Collected Speeds file.

3. Pipeline Order

- Phase 1: Review features / JA and TS feature filtering
- Phase 2: Remove highly correlated features
- Phase 3: Split into train and test datasets
- Phase 4.1: SFS feature selection
- Phase 4.2: Model combinations
- Phase 4.3: RBF scores
- Phase 4.4: SHAP interpretability analysis

Configuration Setup

- Before running the pipeline, I reviewed the config.yaml file because the pipeline depends on this file to know which input files to use, where to save the outputs, and which group labels/classes belong to each dataset.

The original config.yaml [config.yaml](#)

```
data:
  feature_names_file: "data/raw/ISB Abstract - Joint Angles and TS Variable Names.xlsx"
  variant_file: "data/raw/2024 FINAL - Autism Study MAR2024 ALL (AM+NM) Features.xlsx"
  invariant_file: "data/raw/2024 FINAL - Autism Study MAR2024 ALL (AM+NM) Features.xlsx"

  variant_output: "data/processed/autism_2024_train.xlsx"
  invariant_output: "data/processed/autism_2024_test.xlsx"

  sfs_results_excel_file: "src/gaitml/features/autism_2024_phased_1_SFS_results.xlsx"
  sfs_results_file: "src/gaitml/features/autism_2024_phased_1_SFS_results.xlsx"
  combination_results_file: "autism_2024_Step_2_Results_Filtered.xlsx"

  scores_file: "autism_2024_Scores_RBF.xlsx"
  train_file: "data/processed/autism_2024_train.xlsx"
  test_file: "data/processed/autism_2024_test.xlsx"

  output_file: "autism_2024_SHAP_data.xlsx"
  output_dir: "outputs/autism_2024_SHAP"

remove_correlated_features:
  highly_correlated_features_file: "data/raw/Highly correlated features.xlsx"

  input_excel_file: "data/processed/autism_2024_3A_TS_features.xlsx"
  output_excel_file: "data/processed/autism_2024_NH_correlated_features.xlsx"

split_datasets:
  input_file: "data/processed/autism_2024_NH_correlated_features.xlsx"

  output_train_file: "data/processed/autism_2024_train.xlsx"
  output_test_file: "data/processed/autism_2024_test.xlsx"

SFS:
  dataset_train_file: "data/processed/autism_2024_train.xlsx"
  dataset_test_file: "data/processed/autism_2024_test.xlsx"

  Phased_1_SFS_scores_file: "src/gaitml/features/autism_2024_phased_1_SFS_results.xlsx"

  labels_column: "Group"

classes:
  0: "Control"
  1: "Autism"
```

It was mainly prepared for a previous dataset, so I created separate configuration files to avoid mixing paths between studies and to keep the original setup safer.

I created two dataset-specific config files:

- config_autism_2024.yaml

- config_young_old_2024.yaml

The reason for separating them is that Autism 2024 and Young vs Older 2024 use different input files, output file names, and group labels.

For Autism 2024:

- Input file: 2024 FINAL - Autism Study MAR2024 ALL (AM+NM) Features.xlsx
- Group labels: Control and Autism
- Encoding used later in the ML steps: Control = 0, Autism = 1

For Young vs Older 2024:

- Input file: 2024 ALL Post-Processing Speeds Young and Older Adults ALL Features - FINAL AUGUST 20.xlsx
- Group labels: Young and Older
- Encoding used later in the ML steps: Young = 0, Older = 1

During the workflow, I updated `config.yaml` depending on which dataset and which phase I was running. This was important because some later phases use different fields from the config file. For example, Phase 1 uses the raw input and JA/TS output paths, while Phase 4.2 and Phase 4.3 use the train/test files, SFS results, filtered model combinations, and RBF score files.

Configuration note

I updated `config.yaml` depending on the dataset and phase being run. Phase 1 uses the raw input and JA/TS paths, while later phases use train/test files, SFS results, filtered model combinations, and RBF score files.

4. Understanding of Each Step and Result

Phase 1 - Feature Review / JA and TS Filtering

Item	Summary
Purpose	Keep the selected Joint Angle and Temporal-Spatial features from the original dataset.
Autism 2024	The raw Autism file was filtered and produced an Autism JA/TS feature file. The output had fewer columns than the raw file, which means the filtering step worked. At the first we have 260 columns and after phase 1 we have 118 columns
Young vs Older 2024	Only the Young vs Older Post-Processing file was used. The processed JA/TS feature file was created from that file. At the first we have 260 columns and after phase 1 we have 118 columns
Output / result	Autism: autism_2024_JA_TS_features.xlsx. Young vs Older: young_old_2024_JA_TS_features.xlsx.
My understanding	This phase prepares the dataset before ML. It does not train a model; it only keeps the features that should enter the next cleaning step.

Step-wise Results / Notes

Autism 2024:

- Input file: 2024 FINAL - Autism Study MAR2024 ALL (AM+NM) Features.xlsx
- Initial number of columns: 260
- **Output file:** [autism_2024_JA_TS_features.xlsx](#)
[autism_2024_JA_TS_features.xlsx](#)

- Number of columns after Phase 1: 118
- Features kept: Joint Angle and Temporal-Spatial features
- Features removed: Features outside the selected JA/TS feature list
- My note: This step reduced the dataset to the main biomechanical features needed for the ML pipeline.

Young vs Older 2024:

- Input file: 2024 ALL Post-Processing Speeds Young and Older Adults ALL Features - FINAL AUGUST 20.xlsx
- Initial number of columns: 260
- **Output file:** [young_old_2024_JA_TS_features.xlsx](#)
[young_old_2024_JA_TS_features.xlsx](#)
- Number of columns after Phase 1: 118
- Features kept: Joint Angle and Temporal-Spatial features

- Features removed: Features outside the selected JA/TS feature list

So the features dropped are: 142

Phase 1 Dropped Features

Phase 1 reduced the dataset from 260 columns to 118 columns. Total features dropped: 142.

Feature list 1	Feature list 2	Feature list 3	Feature list 4
1. Moment_A1S	2. Moment_A1S_time	3. Power_A2S	4. Power_A2S_time
5. Moment_H1S	6. Moment_H1S_time	7. Power_H1S	8. Power_H1S_time
9. Moment_H2S	10. Moment_H2S_time	11. Power_H2S	12. Power_H2S_time
13. Moment_H3S	14. Moment_H3S_time	15. Power_H3S	16. Power_H3S_time
17. Moment_H1F	18. Moment_H1F_time	19. Moment_H2F	20. Moment_H2F_time
21. Power_K0S	22. Power_K0S_time	23. Moment_K1S	24. Moment_K1S_time
25. Power_K1S	26. Power_K1S_time	27. Moment_K2S	28. Moment_K2S_time
29. Power_K2S	30. Power_K2S_time	31. Moment_K3S	32. Moment_K3S_time
33. Power_K3S	34. Power_K3S_time	35. Power_K4S	36. Power_K4S_time
37. Planar_Time_to_OTO	38. Planar_Time_to_OHS	39. Planar_Time_to_TO	40. Time_to_OTO
41. Time_to_OHS	42. Time_to_TO	43. F2G_ROM	44. F2Ps_ROM
45. S2F_ROM	46. S2G_ROM	47. S2V_ROM	48. V2G_ROM
49. F2Pt_ROM	50. MLA_ROM	51. Cal_Met_Angle_X_ROM	52. Cal_Met_Angle_Y_ROM
53. Cal_Met_Angle_Z_ROM	54. Cal_Mid_Angle_X_ROM	55. Cal_Mid_Angle_Y_ROM	56. Cal_Mid_Angle_Z_ROM
57. Mid_Met_Angle_X_ROM	58. Mid_Met_Angle_Y_ROM	59. Mid_Met_Angle_Z_ROM	60. Sha_Cal_Angle_X_ROM
61. Sha_Cal_Angle_Y_ROM	62. Sha_Cal_Angle_Z_ROM	63. Sha_Cal_Angle_X_HS	64. Sha_Cal_Angle_Y_HS
65. Sha_Cal_Angle_Z_HS	66. Sha_Cal_Angle_X_OTO	67. Sha_Cal_Angle_Y_OTO	68. Sha_Cal_Angle_Z_OTO
69. Sha_Cal_Angle_X_OHS	70. Sha_Cal_Angle_Y_OHS	71. Sha_Cal_Angle_Z_OHS	72. Sha_Cal_Angle_X_TO
73. Sha_Cal_Angle_Y_TO	74. Sha_Cal_Angle_Z_TO	75. Mid_Met_Angle_X_HS	76. Mid_Met_Angle_Y_HS
77. Mid_Met_Angle_Z_HS	78. Mid_Met_Angle_X_OTO	79. Mid_Met_Angle_Y_OTO	80. Mid_Met_Angle_Z_OTO
81. Mid_Met_Angle_X_OHS	82. Mid_Met_Angle_Y_OHS	83. Mid_Met_Angle_Z_OHS	84. Mid_Met_Angle_X_TO
85. Mid_Met_Angle_Y_TO	86. Mid_Met_Angle_Z_TO	87. Cal_Met_Angle_X_HS	88. Cal_Met_Angle_Y_HS
89. Cal_Met_Angle_Z_HS	90. Cal_Met_Angle_X_OTO	91. Cal_Met_Angle_Y_OTO	92. Cal_Met_Angle_Z_OTO
93. Cal_Met_Angle_X_OHS	94. Cal_Met_Angle_Y_OHS	95. Cal_Met_Angle_Z_OHS	96. Cal_Met_Angle_X_TO
97. Cal_Met_Angle_Y_TO	98. Cal_Met_Angle_Z_TO	99. Cal_Mid_Angle_X_HS	100. Cal_Mid_Angle_Y_HS
101. Cal_Mid_Angle_Z_HS	102. Cal_Mid_Angle_X_OTO	103. Cal_Mid_Angle_Y_OTO	104. Cal_Mid_Angle_Z_OTO
105. Cal_Mid_Angle_X_OHS	106. Cal_Mid_Angle_Y_OHS	107. Cal_Mid_Angle_Z_OHS	108. Cal_Mid_Angle_X_TO
109. Cal_Mid_Angle_Y_TO	110. Cal_Mid_Angle_Z_TO	111. F2G_HS	112. F2G_OTO
113. F2G_OHS	114. F2G_TO	115. F2Ps_HS	116. F2Ps_OTO
117. F2Ps_OHS	118. F2Ps_TO	119. F2Pt_HS	120. F2Pt_OTO
121. F2Pt_OHS	122. F2Pt_TO	123. MLA_HS	124. MLA_OTO
125. MLA_OHS	126. MLA_TO	127. S2F_HS	128. S2F_OTO
129. S2F_OHS	130. S2F_TO	131. S2G_HS	132. S2G_OTO

133. S2G_OHS	134. S2G_TO	135. S2V_HS	136. S2V_OTO
137. S2V_OHS	138. S2V_TO	139. V2G_HS	140. V2G_OTO
141. V2G_OHS	142. V2G_TO		

Phase 2 - Remove Highly Correlated Features

Item	Summary
Purpose	Remove redundant features based on the highly correlated feature list.
Autism 2024	The important point for Autism was to make sure Phase 2 uses the Phase 1 JA/TS file, not the raw Autism file. When the raw file was used by mistake, the output had more columns than expected. After correcting the config, the phase continued from the JA/TS output. At the first we have 118 columns and after phase 2 we have 86 columns
Young vs Older 2024	The Young vs Older JA/TS file was used as input, and the non-highly-correlated output file was generated. At the first we have 118 columns and after phase 2 we have 86 columns
Output / result	Autism: autism_2024_NH_correlated_features.xlsx. Young vs Older: young_old_2024_NH_correlated_features.xlsx.
My understanding	This phase reduces redundancy. Features removed here should still be confirmed with the team before they are treated as permanently dropped.

Step-wise Results / Notes

Autism 2024:

- Input file: autism_2024_JA_TS_features.xlsx
- Initial number of columns: 118
- Output file: [autism_2024_NH_correlated_features.xlsx](#)

[autism_2024_NH_correlated_features.xlsx](#)

- Number of columns after Phase 2: 86

Young vs Older 2024:

- Input file: young_old_2024_JA_TS_features.xlsx
- Initial number of columns: 118
- Output file: [young_old_2024_NH_correlated_features.xlsx](#)

[young_old_2024_NH_correlated_features.xlsx](#)

- Number of columns after Phase 2: 86

Dropped features: 32

Feature list 1	Feature list 2	Feature list 3	Feature list 4
1. Hip_Angle_X_MIN_SW_time	2. Stride_Length	3. Speed	4. Stance_Time
5. ND_DOUBLE_LIMB_SUPPORT_TIME	6. ND_SPEED	7. ND_STEP_LENGTH	8. ND_STANCE_TIME
9. ND_STEPS_PER_MINUTE	10. ND_STRIDE_LENGTH	11. ND_SWING_TIME	12. Trunk_Angle_Y_ROM
13. Hip_Angle_X_ROM	14. Pelv_Angle_Y_HS	15. Pelv_Angle_Z_HS	16. Pelv_Angle_X_OTO

17. Pelv_Angle_X_OHS	18. Pelv_Angle_X_TO	19. Pelv_Angle_Y_TO	20. Trunk_Angle_Z_HS
21. Trunk_Angle_X_OTO	22. Trunk_Angle_X_OHS	23. Trunk_Angle_Z_OHS	24. Trunk_Angle_X_TO
25. Trunk_Angle_Y_TO	26. Trunk_Angle_Z_TO	27. Hip_Angle_X_HS	28. Hip_Angle_X_OTO
29. Hip_Angle_X_OHS	30. Hip_Angle_X_TO	31. Hip_Angle_Y_TO	32. Hip_Angle_Z_TO

Phase 3 - Train/Test Split

Item	Summary
Purpose	Split the cleaned dataset into train and test files. 80% train and 20%test
Autism 2024	The Autism non-highly-correlated file was split into training and testing files.
Young vs Older 2024	The Young vs Older non-highly-correlated file was split into training and testing files.
Output / result	Autism: autism_2024_train.xlsx and autism_2024_test.xlsx. Young vs Older: young_old_2024_train.xlsx and young_old_2024_test.xlsx.
My understanding	The train set is used for feature/model selection. The test set is kept for evaluating model performance.

Step-wise Results / Notes

Autism 2024:

- Input file: autism_2024_NH_correlated_features.xlsx

- Output train file: autism_2024_train.xlsx

[autism_2024_train.xlsx](#)

- Output test file: autism_2024_test.xlsx

[autism_2024_test.xlsx](#)

- Split used: 80% train and 20% test

- My note: This step prepared the cleaned Autism dataset for model training and evaluation. The train file is used for feature/model selection, while the test file is used to check model performance on unseen data.

Young vs Older 2024:

- Input file: young_old_2024_NH_correlated_features.xlsx

- Output train file: young_old_2024_train.xlsx

[young_old_2024_train.xlsx](#)

- Output test file: young_old_2024_test.xlsx

[young_old_2024_test.xlsx](#)

- Split used: 80% train and 20% test

- My note: This step prepared the cleaned Young vs Older dataset for the ML phases. The split keeps the evaluation separate from the training process.

Phase 4.1 - SFS Feature Selection

Item	Summary
Purpose	Use Sequential Feature Selection to identify useful feature subsets for the model.
Autism 2024	SFS ran for Autism and produced the Autism SFS results file.
Young vs Older 2024	SFS ran for Young vs Older and produced the Young vs Older SFS results file.
Output / result	Autism: autism_2024_phase4_1_SFS_results.xlsx. Young vs Older: young_old_2024_phase4_1_SFS_results.xlsx.
My understanding	This step gives candidate feature orders/subsets that are then used to build model combinations.

Step-wise Results / Notes

Autism 2024:

- Input files:
 - autism_2024_train.xlsx
 - autism_2024_test.xlsx
- Output file: autism_2024_phase4_1_SFS_results.xlsx

[autism_2024_phase4_1_SFS_results.xlsx](#)

- Purpose: identify ordered candidate feature subsets using Sequential Feature Selection.
- My note: This step does not choose the final model yet. It prepares ranked feature combinations that will be tested in Phase 4.2.

Young vs Older 2024:

- Input files:
 - young_old_2024_train.xlsx
 - young_old_2024_test.xlsx
- Output file: young_old_2024_phase4_1_SFS_results.xlsx

[young_old_2024_phase4_1_SFS_results.xlsx](#)

- Purpose: identify ordered candidate feature subsets using Sequential Feature Selection.
- My note: The SFS output is used as the base for generating model combinations in the next step.

Phase 4.2 - Model Combinations

Item	Summary
Purpose	Generate and evaluate many SVM/RBF model combinations using the SFS feature subsets.
Autism 2024	The Autism Step 2 result file was generated and filtered. The best CV accuracy I observed was around 0.83, so the 0.90 threshold was too strict for this dataset. I used a lower threshold and filtered by number of features, test accuracy, and sensitivity.
Young vs Older 2024	The Young vs Older Step 2 result file was also generated and filtered using the same logic.
Output / result	Autism: autism_2024_Step_2_Results_Filtered.xlsx. Young vs Older: young_old_2024_Step_2_Results_Filtered.xlsx.
My understanding	This phase creates many candidate models. Filtering is needed before moving forward so that weak or irrelevant models do not dominate later interpretation.

Step-wise Results / Notes

Autism 2024:

- Input files:

- autism_2024_train.xlsx
- autism_2024_test.xlsx
- autism_2024_phase4_1_SFS_results.xlsx

- Output file: autism_2024_Step_2_Results_Filtered.xlsx

[autism_2024_Step_2_Results_Filtered.xlsx](#)

- Purpose: test many SVM/RBF model combinations using the feature subsets from SFS.

- Filtering decision: The original threshold CV_Accuracy ≥ 0.90 was too high for Autism because the best CV_Accuracy was around 0.83. I lowered the threshold to keep meaningful candidate models.

- Model selection criteria used:

- CV_Accuracy
- Test_Accuracy
- Sensitivity
- #Features

- My note: This step helps shortlist candidate models before final scoring. For Autism, I had to adjust the filtering threshold based on the actual results instead of forcing 0.90.

Young vs Older 2024:

- Input files:

- young_old_2024_train.xlsx
- young_old_2024_test.xlsx
- young_old_2024_phase4_1_SFS_results.xlsx

- Output file: young_old_2024_Step_2_Results_Filtered.xlsx

[young_old_2024_Step_2_Results_Filtered.xlsx](#)

- Purpose: generate and evaluate SVM/RBF model combinations using the SFS feature subsets.
- Model selection criteria used:
 - CV_Accuracy
 - Test_Accuracy
 - Sensitivity
 - #Features
- My note: I applied the same filtering logic to keep the strongest candidate models for the next scoring step.

Code adjustment note:

- In [get_model_combinations_with_accuracy.py](#), I updated the Group label encoding so the model could work with both datasets.
 - For Autism, the labels were mapped as Control = 0 and Autism = 1.
 - For Young vs Older, the labels were mapped as Young = 0 and Older = 1.
- I also cleaned label text using `.str.strip()` and added a check for unmapped labels.

RESULTS:

[results/_Step_2_Results_Filtered.xlsx.xlsx](#)

FOR AUTISM :

-5 best model for autism

Name_Model	CV_Accuracy	Test_Accuracy	Sensitivity	#Features	C	gamma	CV_split	Random_State	Feature_idx	Confusion_Matrix	y_pred
417	0.838983051	0.566666667	0.61538462	16	1	0.1	10	0	0	[[9 8] [5 8]]	[0 1 1 1 0 1 0 0 1 0 1 1 0 1 1 1 1 0 0 0 1 1 0 0 1 0 1 1 0 0]
464	0.830508475	0.633333333	0.61538462	17	1	0.1	10	0	0	[[11 6] [5 8]]	[0 1 1 1 0 1 0 0 0 0 1 1 0 0 1 1 1 0 0 0 1 1 0 0 1 0 1 1 0 0]
531	0.830508475	0.566666667	0.61538462	18	1	0.1	10	0	0	[[9 8] [5 8]]	[0 1 1 1 0 1 0 0 1 0 1 1 0 1 1 1 1 0 0 0 1 1 0 0 1 0 1 1 0 0]
599	0.822033898	0.633333333	0.61538462	19	1	0.1	10	0	0	[[11 6] [5 8]]	[0 1 1 1 0 1 0 0 0 0 1 1 0 0 1 1 1 0 0 0 1 1 0 0 1 0 1 1 0 0]
250	0.822033898	0.566666667	0.61538462	13	1	0.1	10	0	0	[[9 8] [5 8]]	[0 1 1 1 0 1 0 0 1 0 1 1 0 1 0 1 1 0 0 0 1 1 0 0 1 0 1 1 1 0]

-best model for 5 features:

Name_Model	CV_Accuracy	Test_Accuracy	Sensitivity	#Features	C	gamma	CV_split	Random_State	Feature_idx	Confusion_Matrix	y_pred
1260	0.779561017	0.666666667	0.59230769	5	30	0.05	10	0	2	[[11 6] [4 9]]	[0 1 1 1 0 1 1 1 0 1 0 0 0 1 0 1 1 0 1 1 0 0 0 1 0 1 1 0 0 0]

-best model for 6 features:

Name_Model	CV_Accuracy	Test_Accuracy	Sensitivity	#Features	C	gamma	CV_split	Random_State	Feature_idx	Confusion_Matrix	y_pred
1308	0.796610169	0.633333333	0.76923077	6	20	0.1	10	0	2	[[9 8] [3 10]]	[0 1 1 1 1 1 1 1 0 1 0 0 0 1 0 1 1 0 1 1 0 1 0 1 0 1 1 1 0 0]

-best model for 10 features:

Name_Model	CV_Accuracy	Test_Accuracy	Sensitivity	#Features	C	gamma	CV_split	Random_State	Feature_idx	Confusion_Matrix	y_pred
3548	0.813559322	0.7	0.61538462	10	10	0.5	10	0	5	[[13 4] [5 8]]	[0 1 0 0 1 0 0 0 0 1 0 0 0 0 0 1 1 1 1 1 0 0 1 0 0 1 1 0 0]

-best model for 15 features:

Name_Model	CV_Accuracy	Test_Accuracy	Sensitivity	#Features	C	gamma	CV_split	Random_State	Feature_idx	Confusion_Matrix	y_pred
358	0.822033898	0.566666667	0.61538462	15	1	0.1	10	0	0	[[9 8] [5 8]]	[0 1 1 1 0 1 0 0 1 0 1 1 0 1 1 1 1 0 0 0 1 1 0 0 1 0 1 1 0 0]

FOR YOUNG /OLD :

-5 best model

Name_Model	CV_Accuracy	Test_Accuracy	Sensitivity	#Features	C	gamma	CV_split	Random_State	Feature_idx	Confusion_Matrix	y_pred
7603	0.935483871	0.9375	0.875	7	10	0.5	10	0	4 [[8 0] [1 7]]	[1 0 1 1 0 1 0 1 0 0 0 0 0 1 0 1]	
7610	0.935483871	0.9375	0.875	7	20	0.5	10	0	4 [[8 0] [1 7]]	[1 0 1 1 0 1 0 1 0 0 0 0 0 1 0 1]	
7617	0.935483871	0.9375	0.875	7	30	0.5	10	0	4 [[8 0] [1 7]]	[1 0 1 1 0 1 0 1 0 0 0 0 0 1 0 1]	
7625	0.935483871	0.9375	0.875	7	40	0.5	10	0	4 [[8 0] [1 7]]	[1 0 1 1 0 1 0 1 0 0 0 0 0 1 0 1]	
7633	0.935483871	0.9375	0.875	7	50	0.5	10	0	4 [[8 0] [1 7]]	[1 0 1 1 0 1 0 1 0 0 0 0 0 1 0 1]	

-best model for 5 features:

Name_Model	CV_Accuracy	Test_Accuracy	Sensitivity	#Features	C	gamma	CV_split	Random_State	Feature_idx	Confusion_Matrix	y_pred
4021	0.935483871	0.75	0.75	5	20	0.5	10	0	2	[[62] [26]]	[1010100100101101]

-best model for 6 features:

Name_Model	CV_Accuracy	Test_Accuracy	Sensitivity	#Features	C	gamma	CV_split	Random_State	Feature_idx	Confusion_Matrix	y_pred
7536	0.919354839	0.8125	0.875	6	100	0.1	10	0	4	[[6 2] [1 7]]	[1 0 1 0 0 1 0 1 0 1 1 1 1]

-best model for 10 features:

Name_Model	CV_Accuracy	Test_Accuracy	Sensitivity	#Features	C	gamma	CV_split	Random_State	Feature_idx	Confusion_Matrix	y_pred
8012	0.935483871	0.75	0.75	10	10	0.5	10	0	4	[[6 2] [2 6]]	[1 1 1 1 0 0 1 0 0 0 0 0 1 0 1]

-best model for 15 features:

Name_Model	CV_Accuracy	Test_Accuracy	Sensitivity	#Features	C	gamma	CV_split	Random_State	Feature_idx	Confusion_Matrix	y_pred
6764	0.919354839	0.75	0.625	15	10	0.05	10	0	3	[[7 1] [3 5]]	[1 0 1 1 1 0 0 1 0 0 0 0 0 0 1]

Phase 4.3 - RBF Scores

Item	Summary
Purpose	Use the filtered Step 2 results to calculate the final RBF score table.
Autism 2024	The Autism Scores RBF file was generated successfully.
Young vs Older 2024	The Young vs Older Scores RBF file was generated successfully.
Output / result	Autism: autism_2024_Scores_RBF.xlsx. Young vs Older: young_old_2024_Scores_RBF.xlsx.
My understanding	This file helps shortlist the strongest models using performance scores such as CV accuracy, test accuracy, and sensitivity.

Output : autism_2024_Scores_RBF.xlsx. Young vs Older:

[autism_2024_Scores_RBF.xlsx](#)

output: young_old_2024_Scores_RBF.xlsx.

[young_old_2024_Scores_RBF.xlsx](#)

FOR AUTISM :

-5 best model for autism

Name_Model	#Features	C_Value	Gamma_Value	Feature_Id	tol	CV_accuracy	Test_Accuracy	Specificity	Sensitivity	NPV	PPV	Likelihood_Ratio	F1	MCC	Confusion_Matrix	Features	CV_split	Random_State
417	16	1	0.1	0	0.001	0.838983051	0.566666667	0.5294118	0.6153846	0.64286	0.5	0.726495726	0.55172	0.14382	0	[28 53 9 79 42 26 39 21 12 51 / 41 24 59 13 17]	[19 8][5 8]	[0 1 1 1 0 1 0 0 1 1 0 1 1 1 0 0 0]
464	17	1	0.1	0	0.001	0.830208475	0.533333333	0.6470588	0.6153846	0.6875	0.57143	0.59440594	0.59259	0.26068	0	[28 53 9 79 42 26 39 21 12 51 / 41 24 59 13 17 15]	[11 6][5]	[0 1 1 1 0 1 0 0 0 1 1 0 0 1 1 0 0 0]
531	18	1	0.1	0	0.001	0.830508475	0.566666667	0.5294118	0.6153846	0.64286	0.5	0.726495726	0.55172	0.14382	0	[28 53 9 79 42 26 39 21 12 51 / 41 24 59 13 17 15 76]	[19 8][5 8]	[0 1 1 1 0 1 0 0 1 1 0 1 1 1 1 0 0 0]
599	19	1	0.1	0	0.001	0.822033898	0.633333333	0.6470588	0.6153846	0.6875	0.57143	0.59440594	0.59259	0.26068	0	[28 53 9 79 42 26 39 21 12 51 / 41 24 59 13 17 15 76 0]	[11 6][5]	[0 1 1 1 0 1 0 0 0 1 1 0 0 1 1 1 0 0 0]
250	13	1	0.1	0	0.001	0.822033898	0.566666667	0.5294118	0.6153846	0.64286	0.5	0.726495726	0.55172	0.14382	0	[28 53 9 79 42 26 39 21 12 51 / 41 24]	[19 8][5 8]	[0 1 1 1 0 1 0 0 1 1 0 1 1 0 1 1 0 0 0]

-best model for 5 features:

Name_Model	#Features	C_Value	Gamma_Value	Feature_Id	tol	CV_accuracy	Test_Accuracy	Specificity	Sensitivity	NPV	PPV	Likelihood_Ratio	F1	MCC	Confusion_Matrix	Features	CV_split	Random_State
1250	5	30	0.05	2	0.001	0.779661017	0.666666667	0.61705882	0.62230769	0.73333	0.6	0.475524176	0.61286	0.33634	0	[28 4 36 21 30]	[11 6][5]	[0 1 1 1 0 1 1 1 0 1 0 0 0 1 0]

num features

;28,4,36,21,30

-best model for 6 features:

Name_Model	#Features	C_Value	Gamma_Value	Feature_Id	tol	CV_accuracy	Test_Accuracy	Specificity	Sensitivity	NPV	PPV	Likelihood_Ratio	F1	MCC	Confusion_Matrix	Features	CV_split	Random_State
1308	6	20	0.1	2	0.001	0.796610167	0.633333333	0.52941176	0.76923077	0.75	0.55556	0.435897436	0.54536	0.30208	0	[28 4 36 21 30 8]	[19 8][5]	[0 1 1 1 1 1 1 1 0 1 0 0 0 1 0]

-best model for 10 features:

Name_Model	#Features	C_Value	Gamma_Value	Feature_Id	tol	CV_accuracy	Test_Accuracy	Specificity	Sensitivity	NPV	PPV	Likelihood_Ratio	F1	MCC	Confusion_Matrix	Features	CV_split	Random_State
3648	10	10	0.5	5	0.001	0.813559122	0.7	0.76470588	0.61538462	0.72222	0.66667	0.50225858	0.64	0.38446	0	[28 26 4 8 21 36 3 11 20 51]	[13 4][5]	[0 1 0 1 0 1 0 0 0 1 0 0 0 0 0]

-best model for 15 features:

Name_Model	#Features	C_Value	Gamma_Value	Feature_Id	tol	CV_accuracy	Test_Accuracy	Specificity	Sensitivity	NPV	PPV	Likelihood_Ratio	F1	MCC	Confusion_Matrix	Features	CV_split	Random_State
368	15	1	0.1	0	0.001	0.822033898	0.566666667	0.52941176	0.61538462	0.64286	0.5	0.726495726	0.55172	0.14382	0	[28 53 9 79 42 26 39 21 12 51 / 41 24 59 13]	[19 8][5 8]	[0 1 1 1 0 1 0 1 1 0 1 1 0 1 1]

Phase 4.4 - SHAP Interpretability

Item	Summary
Purpose	Explain the selected models and identify which features contribute most to model decisions.
Autism 2024	SHAP was started for Autism. I fixed/handled setup issues such as installing SHAP, sheet naming expected by the code, and numeric group encoding. The run then started, but it attempted to process 4,191 models, which would take a very long time.
Young vs Older 2024	Young vs Older SHAP has not been finalized yet. It should be run after the team confirms how many top models should be selected.
Output / result	Expected outputs: SHAP Excel data and SHAP plots for selected models.
My understanding	SHAP should probably be run only on the top selected models, such as top 3 or top 5, instead of all models. I asked Karansinh and Ashirbad to confirm this.

5. Main Files Generated So Far

Dataset	Generated files
Autism 2024	autism_2024_JA_TS_features.xlsx autism_2024_NH_correlated_features.xlsx autism_2024_train.xlsx autism_2024_test.xlsx autism_2024_phase4_1_SFS_results.xlsx autism_2024_Step_2_Results_Filtered.xlsx autism_2024_Scores_RBF.xlsx
Young vs Older 2024	young_old_2024_JA_TS_features.xlsx young_old_2024_NH_correlated_features.xlsx young_old_2024_train.xlsx young_old_2024_test.xlsx young_old_2024_phase4_1_SFS_results.xlsx young_old_2024_Step_2_Results_Filtered.xlsx young_old_2024_Scores_RBF.xlsx

6. Current Notes and Issues

- The original config file was not fully prepared for the two current datasets, so I had to update paths carefully depending on the phase and dataset.
- For Autism, the Phase 4.2 filtering showed that 0.90 was too strict because the best CV accuracy was around 0.83. I adjusted the filtering threshold to keep meaningful candidate models.

7. Current Interpretation

Both datasets successfully reached Phase 4.3. This means the pipeline can process the raw datasets, filter features, remove redundant features, split data, run SFS, generate model combinations, and calculate RBF scores.

8. Next Steps

- Run Phase 4.4 SHAP for Autism and Young vs Older using only the approved selected models.
- Prepare SHAP feature ranking tables and plots.
- Summarize which features are important, which features look redundant, and which decisions need team confirmation before dropping features.

Jun 2, 2026

XGBoost Pipeline Update

Dataset: Young vs Older 2024 Dataset

Today, I started working on an experimental version of the ML pipeline using **XGBoost** instead of the original SVM RBF model. The goal was to compare the performance of XGBoost with the existing SVM pipeline, while keeping the original SVM files unchanged.

First, I created a separate copy of the project folder named **Gait-ML-Shap-main_y_Mohamed**. This allowed me to safely modify and test the XGBoost version without affecting the original SVM pipeline.

Then, I created a new pipeline folder for XGBoost and started preparing separate files for the new model. I also added new output paths in the config.yaml file [config.yaml](#)

```
data:
  feature_names_file: "data/raw/ISB Abstract - Joint Angles and TS Variable Names.xlsx"
  variant_file: "data/raw/2024 FINAL - Autism Study MAR2024 ALL (AM+NM) Features.xlsx"
  invariant_file: "data/raw/2024 FINAL - Autism Study MAR2024 ALL (AM+NM) Features.xlsx"

  variant_output: "results/autism_2024_JA_TS_features.xlsx"
  invariant_output: "results/autism_2024_JA_TS_features.xlsx"
  sfs_results_excel_file_xgboost: "results/autism_2024_phase4_1_SFS_results_XGBoost.xlsx"
  combination_results_file_xgboost: "results/autism_2024_Step_2_Results_XGBoost.xlsx"
  scores_file_xgboost: "results/autism_2024_Scores_XGBoost.xlsx"

remove_correlated_features:
  highly_correlated_features_file: "data/raw/Highly correlated features.xlsx"

  input_excel_file: "results/autism_2024_JA_TS_features.xlsx"
  output_excel_file: "results/autism_2024_NH_correlated_features.xlsx"

split_datasets:
  input_file: "results/autism_2024_NH_correlated_features.xlsx"

  output_train_file: "results/autism_2024_train.xlsx"
  output_test_file: "results/autism_2024_test.xlsx"

SFS:
  dataset_train_file: "results/autism_2024_train.xlsx"
  dataset_test_file: "results/autism_2024_test.xlsx"

  Phase4_1_SFS_scores_file: "results/autism_2024_phase4_1_SFS_results_XGBoost.xlsx"

  labels_column: "Group"

classes:
  0: "Control"
  1: "Autism"
```

- To avoid changing the original configuration directly, I organized the configuration into separate files. I kept the original `config.yaml` as the general/default config, then created two additional config files: one for Autism 2024 and one for Young vs Older 2024.

The reason for doing this is that each dataset has different input files, output names, class labels, and generated results. For example, Autism uses `Control` and `Autism` as groups, while Young vs Older uses `Young` and `Older`. So having separate config files makes the workflow cleaner and reduces the risk of mixing paths between datasets.

The two new config files are:

- config_autism_2024.yaml
- config_young_old_2024.yaml

When I want to run the pipeline for a specific dataset, I use the corresponding config values for that dataset. This helped me run the same pipeline structure for both datasets while keeping the original setup safe and easier to track.

XGBoost Model-Combination Search Space

Parameter	Values tested
Number of selected features	5 to 19
n_estimators	50, 100, 150
max_depth	2, 3, 4
learning_rate	0.01, 0.05, 0.1
subsample	0.8, 1.0
colsample_bytree	0.8, 1.0
Cross-validation	10-fold cross-validation

Early comparison

Early results showed XGBoost reaching around 0.95 CV accuracy, while the best SVM RBF CV accuracy was around 0.93. This suggests that XGBoost may be promising, but the final decision should use complete metrics: test accuracy, sensitivity, specificity, F1 score, and MCC.

This creates many model combinations, so Phase 4.2 takes a long time to run. While it was running, I observed that the process was progressing through different Feature_idx values and different numbers of selected features.

I also compared the early results with the original SVM RBF model. The best CV accuracy observed with XGBoost reached around 0.95, while the best CV accuracy from the SVM RBF model was around 0.93. This suggests that XGBoost may be promising, but the final decision should depend on the complete results, including test accuracy, sensitivity, specificity, F1 score, and MCC.

At the end of the day, I stopped while Phase 4.2 was still running/completing the XGBoost model combinations.

Output files created/planned:

1- results/young_old_2024_JA_TS_features.xlsx [young_old_2024_JA_TS_features.xlsx](#)

2- results/young_old_2024_NH_correlated_features.xlsx

[young_old_2024_NH_correlated_features.xlsx](#)

3- results/young_old_2024_train.xlsx [young_old_2024_train.xlsx](#)

4- results/young_old_2024_test.xlsx [young_old_2024_test.xlsx](#)

5- results/young_old_2024_phase4_1_SFS_results_XGBoost.xlsx

[young_old_2024_phase4_1_SFS_results_XGBoost.xlsx](#)

6- results/young_old_2024_Step_2_Results_XGBoost.xlsx

[young_old_2024_Step_2_Results_XGBoost.xlsx](#)

7- results/young_old_2024_Scores_XGBoost.xlsx [young_old_2024_Scores_XGBoost.xlsx](#)

RESULTS:

[results/2024_Step_2_Results_XGBoost.xlsx](#)

FOR AUTISM :

-5 best model for autism

	Name_Model	CV_Accuracy	Test_Accuracy	Sensitivity	Specificity	#feature	n_estimator	max_depth	learning_rate	subsample	colsample_bytree	CV_split	Random_State	Feature_idx	Confusion_Matrix	y_pred
1	4021	0.872881356	0.833333333	0.8823529	0.7692308	12	150	4	0.05	0.8	0.8	10	0	3	[[10 3][2 15]]	[1010100111011000110110110011]
2	3985	0.86440678	0.833333333	0.8823529	0.7692308	12	100	4	0.05	0.8	0.8	10	0	3	[[10 3][2 15]]	[1010100111011000110110110011]
4	8542	0.855932203	0.833333333	0.8823529	0.7692308	8	150	4	0.05	0.8	1	10	0	7	[[10 3][2 15]]	[11101001101011000110110110011]
5	8754	0.855932203	0.833333333	0.8823529	0.7692308	10	150	4	0.05	0.8	1	10	0	7	[[10 3][2 15]]	[1010100111011000110110110011]
6	8396	0.855932203	0.8	0.8823529	0.6923077	7	100	3	0.1	1	1	10	0	7	[[9 4][2 15]]	[11111001101011000110110110011]

-best model for 5 features:

	Name_Model	CV_Accurac	Test_Accurac	Sensitivity	Specificit	#Feature	n_estimators	max_depth	learning_rate	subsample	colsample_bytre	CV_split	Random_State	Feature_idx	Confusion_Matri	y_pred
2	8231	0.838983051	0.733333333	0.8235294	0.6153846	5	150	4	0.1	0.8	0.8	10	0	7 [[8 5] [3 14]]	[011110011110111000110110110011]	

-best model for 6 features:

	Name_Model	CV_Accurac	Test_Accurac	Sensitivity	Specificit	#Feature	n_estimators	max_depth	learning_rate	subsample	colsample_bytre	CV_split	Random_State	Feature_idx	Confusion_Matrix	y_pred
607		8293	0.855932203	0.766666667	0.8235294	0.6923077	6	100	3	0.1	1	1	10	0	7 [[9 4][3 14]]	[111110011010111000010110110011]

-best model for 10 features:

	Name_Model	CV_Accurac	Test_Accurac	Sensitivity	Specificit	#Feature	n_estimator	max_depth	learning_rate	subsample	colsample_bytre	CV_split	Random_State	Feature_idx	Confusion_Matri	y_pred
2937	8754	0.855932203	0.833333333	0.8823529	0.7692308	10	150	4	0.05	0.8	1	10	0	7	[[10 3][2 15]]	[10101001110111000110110110011]

-best model for 15 features:

	Name_Model	CV_Accurac	Test_Accurac	Sensitivity	Specificit	#Feature	n_estimator	max_depth	learning_rate	subsample	colsample_bytre	CV_split	Random_State	Feature_idx	Confusion_Matri	y_pred
6317	6851	0.838983051	0.766666667	0.9411765	0.5384615	15	100	3	0.1	1	0.8	10	0	5	[[7 6][1 16]]	[111110111110111010110110110011]

FOR YOUNG /OLD :

-5 best model

#	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
1	Name_Model	CV_Accuracy	Test_Accuracy	Sensitivity	Specificity	#Features	n_estimators	max_depth	learning_rate	subsample	colsample_bytree	CV_split	Random_State	Feature_ids	Confusion_Matrix	y_pred	
2	10833	0.951612903	1	1	1	11	150	3	0.1	0.8	0.8	10	0	7 [[8 0] [0 8]]	[0 1 0 0 1 0 1 0 1 1 0 1 0]		
3	10844	0.951612903	1	1	1	11	150	4	0.1	0.8	0.8	10	0	7 [[8 0] [0 8]]	[0 1 0 0 1 0 1 0 1 1 0 1 0]		
4	10823	0.951612903	1	1	1	11	150	2	0.1	0.8	1	10	0	7 [[8 0] [0 8]]	[0 1 0 0 1 0 1 0 1 1 0 1 0]		
5	10810	0.935483871	1	1	1	11	100	4	0.1	0.8	0.8	10	0	7 [[8 0] [0 8]]	[0 1 0 0 1 0 1 0 1 1 0 1 0]		
6	10822	0.935483871	1	1	1	11	150	2	0.1	0.8	0.8	10	0	7 [[8 0] [0 8]]	[0 1 0 0 1 0 1 0 1 1 0 1 0]		

-best model for 5 features:

Name_Model	CV_Accuracy	Test_Accuracy	Sensitivity	Specificity	#Features	n_estimators	max_depth	learning_rate	subsample	colsample_bytree	CV_split	Random_State	Feature_Ids	Confusion_Matrix	y_pred
1712	0.887096774	0.9375	1	0.875	5	150	3	0.1	1	1	10	0	1 [[7 1] [0 8]]	[0 1 0 0 1 1 1 0 1 1 0 1 0]	

-best model for 6 features:

Name_Model	CV_Accuracy	Test_Accuracy	Sensitivity	Specificity	#Features	n_estimators	max_depth	learning_rate	subsample	colsample_bytree	CV_split	Random_State	Feature_idx	Confusion_Matrix	y_pred
10329	0.903225806	0.875	1	0.75	6	150	2	0.1	0.8	1	10	0	7 [[6 2] [0 8]]	[1 1 0 1 1 0 1 0 1 1 0 1 0]	

-best model for 10 features:

Name_Model	CV_Accuracy	Test_Accuracy	Sensitivity	Specificity	#Features	n_estimators	max_depth	learning_rate	subsample	colsample_bytree	CV_split	Random_State	Feature_ids	Confusion_Matrix	y_pred
10691	0.919354839	0.9375	1	0.875	10	100	100	3	0.05	0.8	0.8	10	0	7 [[7 1] [0 8]]	[1 1 0 0 1 0 1 0 1 1 0 1 0]

-best model for 15 features:

Name_Model	CV_Accuracy	Test_Accuracy	Sensitivity	Specificity	#Features	n_estimators	max_depth	learning_rate	subsample	colsample_bytree	CV_split	Random_State	Feature_ids	Confusion_Matrix	y_pred
11221	0.919354839	0.9375	1	0.875	15	100	100	3	0.1	0.8	1	10	0	7 [[7 1] [0 8]]	[0 1 0 1 1 0 1 0 1 1 0 1 0]

Jun 5, 2026

Today, I helped Ben with the V3D work. Most of the day was focused on verifying the diabetics dataset and checking that the data was correct before continuing with the analysis.

The main task was to review the V3D-exported data carefully and make sure the dataset structure, feature values, and organization were consistent. This step is important because any issue in the raw or exported data can affect the later ML pipeline results.

The priority was to support Ben and make sure the diabetics data was ready and reliable for the next steps.

Jun 8, 2026

SHAP Analysis, XGBoost Comparison, MLflow Tracking, and Time-Variable Raw Data Check

starting from the SVM SHAP review, continuing with the XGBoost SHAP implementation and MLflow logging, and ending with the raw-data/statistics/scatter-plot validation for time-related top features.

Main area	What was done	Main outputs
SVM Top 3 / SHAP	Reviewed the existing Top 3 SVM/RBF models and their SHAP results.	SVM top features and SHAP plots for Young/Old and Autism.
XGBoost Top 3	Created/used Top 3 XGBoost model files after sorting by CV_accuracy, Test_Accuracy, and Sensitivity.	young_old_2024_Scores_XGBoost_Top3.xlsx and autism_2024_Scores_XGBoost_Top3.xlsx.
XGBoost SHAP	Implemented a modular XGBoost SHAP workflow and generated SHAP outputs.	XGBoost SHAP Excel files and summary plot PNGs.
MLflow	Logged XGBoost artifacts, metrics, and SHAP plots.	Autism and Young/Old MLflow runs.
Raw-data check	Checked top time-related features using descriptive statistics and scatter plots.	SVM and XGBoost time-variable stats files and scatter plots.
Discussion with Dr. Vicky	Prepared a short explanation for why XGBoost features differ from SVM.	Short summary message for Slack/discussion.

1. Starting Point and Objective

The work started from the need to analyze the best models and top features for both datasets. The goal was not only to identify high-performing models, but also to understand whether the features selected by the models are meaningful in the raw biomechanical data.

- Dataset 1: Young vs Older 2024.
- Dataset 2: Autism 2024.
- Models compared: SVM/RBF and XGBoost.
- Main validation focus requested by Dr. Vicky: raw data and statistics for the time-related variables among the top features.

2. Top 3 Model / Top Feature File Preparation

Before running SHAP or raw-data checks, the best models had to be identified. The Top 3 files were created by sorting the score files using the following priority:

1. CV_accuracy
2. Test_Accuracy
3. Sensitivity

This sorting strategy was used to keep the strongest models first, while also considering both cross-validation performance and test-set behavior.

1-

autism_2024_Scores_RBF_Top3.xlsx

Name	Model	#Features	C Value	Gamma Value	Feature id	tol	CV accuracy	Test Accuracy	Specificity	Sensitivity	NPV	PPV	Likelihood Ratio	F1	MCC	Confusion Matrix	Features	CV split	Random State
417		16	1	0.1	0	RBF	0.838983051	0.566666667	0.529411765	0.615384615	0.642857	0.5	0.726495726	0.551724	0.143823		0 [28 53 9 79] [19 8] [5 8]] [0 11 10 10 10 10 11 10 00 11 00 01 10 01 10 0]		
464		17	1	0.1	0	RBF	0.830508475	0.633333333	0.647058824	0.615384615	0.6875	0.571429	0.594405594	0.592593	0.26068		0 [28 53 9 79] [11 6] [5 8]] [0 11 10 10 00 01 10 01 11 00 01 10 01 10 0]		
531		18	1	0.1	0	RBF	0.830508475	0.566666667	0.529411765	0.615384615	0.642857	0.5	0.726495726	0.551724	0.143823		0 [28 53 9 79] [19 8] [5 8]] [0 11 10 10 01 10 10 11 10 00 11 00 01 10 0]		

young_old_2024_Scores_RBF_Top3.xlsx

Name	Model	#Features	C Value	Gamma Value	Feature id	tol	CV accuracy	Test Accuracy	Specificity	Sensitivity	NPV	PPV	Likelihood Ratio	F1	MCC	Confusion Matrix	Features	CV split	Random State
7603		7	10	0.5	4	RBF	0.935483871	0.9375	1	0.875	0.88889	1	0.125	0.93333	0.88192		0 [19 22 81 1] [8 0] [1 7]] [1 0 1 1 0 1 0 1 0 0 0 0 0 0 1 0 1]		
7610		7	20	0.5	4	RBF	0.935483871	0.9375	1	0.875	0.88889	1	0.125	0.93333	0.88192		0 [19 22 81 1] [8 0] [1 7]] [1 0 1 1 0 1 0 1 0 0 0 0 0 0 1 0 1]		
7617		7	30	0.5	4	RBF	0.935483871	0.9375	1	0.875	0.88889	1	0.125	0.93333	0.88192		0 [19 22 81 1] [8 0] [1 7]] [1 0 1 1 0 1 0 1 0 0 0 0 0 0 1 0 1]		

2-

autism_2024_Scores_XGBoost_Top3.xlsx

Name	Model	#Features	# estimators	max depth	learning rate	subsample	colsample bytree	CV split	Random State	Feature id	Model Type	CV accuracy	Test Accuracy	Specificity	Sensitivity	NPV	PPV	Likelihood Ratio	F1	MCC	Confusion Matrix	Features
4021		12	150	4	0.05	0.8	0.8	10	0	0	3 XGBoost	0.872881396	0.833333333	0.769230769	0.882352941	0.833333	0.833333	0.152941176	0.857483	0.659082	[[10 3] [2 15]]	[5] 27 26 35 40 4 71 29 44 76 65 24]
3985		12	100	4	0.05	0.8	0.8	10	0	0	3 XGBoost	0.86440678	0.833333333	0.769230769	0.882352941	0.833333	0.833333	0.152941176	0.857483	0.659082	[[10 3] [2 15]]	[5] 27 26 35 40 4 71 29 44 76 65 24]
8542		8	150	4	0.05	0.8	1	10	0	0	7 XGBoost	0.855912203	0.833333333	0.769230769	0.882352941	0.833333	0.833333	0.152941176	0.857483	0.659082	[[10 3] [2 15]]	[26 4 75 6 53 29 24 44]

young_old_2024_Scores_XGBoost_Top3.xlsx

Name	Model	#Features	# estimators	max depth	learning rate	subsample	colsample bytree	CV split	Random State	Feature id	Model Type	CV accuracy	Test Accuracy	Specificity	Sensitivity	NPV	PPV	Likelihood Ratio	F1	MCC	Confusion Matrix	Features
10833		11	150	3	0.1	0.8	0.8	10	0	0	7 XGBoost	0.951612903	1	1	1	1	1	0	1	1	1 [8 0] [0 8]]	[19 67 18 22 56 52 68 72 31 6 44]
10844		11	150	4	0.1	0.8	0.8	10	0	0	7 XGBoost	0.951612903	1	1	1	1	1	0	1	1	1 [8 0] [0 8]]	[19 67 18 22 56 52 68 72 31 6 44]
10823		11	150	2	0.1	0.8	1	10	0	0	7 XGBoost	0.951612903	1	1	1	1	1	0	1	1	1 [8 0] [0 8]]	[19 67 18 22 56 52 68 72 31 6 44]

Model type	Dataset	Top 3 input score file	Top 3 output file
SVM/RBF	Young vs Older	young_old_2024_Scores_RBF.xlsx	young_old_2024_Scores_RBF_Top3.xlsx
SVM/RBF	Autism	autism_2024_Scores_RBF.xlsx	autism_2024_Scores_RBF_Top3.xlsx
XGBoost	Young vs Older	results/young_old_2024_Scores_XGBoost.xlsx	results/young_old_2024_Scores_XGBoost_Top3.xlsx
XGBoost	Autism	results/autism_2024_Scores_XGBoost.xlsx	results/autism_2024_Scores_XGBoost_Top3.xlsx

Important implementation detail

For XGBoost, the selected features were stored as feature indices in the Features column. These indices were converted back into feature names using the feature columns from the train/test files.

3. SVM/RBF SHAP Analysis

The SVM/RBF SHAP analysis was reviewed first because it was the existing reference pipeline. The Top 3 SVM models were used to identify the most important features in the best SVM/RBF models.

- For Young vs Older, the label mapping is Young = 0 and Older = 1.
- For Autism, the label mapping is Control = 0 and Autism = 1.
- The SHAP bar plots show mean absolute SHAP values, so they represent feature importance magnitude rather than direction.
- The top SVM features were then used to identify features containing _time.

The purpose of this step was to know which time variables should be checked in the raw data and in scatter plots.

4. XGBoost SHAP Implementation

After the SVM SHAP review, the same type of SHAP analysis was implemented for XGBoost. The initial code was large, so it was separated into a clean pipeline file and a reusable model utility file.

File	Role
pipelines_xgboost/phase4_4_SHAP_XGBoost.py	Small runner file. It reads config.yaml and calls the XGBoost SHAP function.
src/gaitml/models/get_SHAP_data_XGBoost.py	Main SHAP logic. It reads the Top 3 file, trains XGBoost models, calculates SHAP values, creates Excel outputs, and saves SHAP summary plots.

This structure made the pipeline easier to maintain and closer to the style of the existing SVM pipeline.

Model type	Dataset	Top 3 input score file	Top 3 output file
SVM/RBF	Young vs Older	young_old_2024_Scores_RBF.xlsx	young_old_2024_Scores_RBF_Top3.xlsx
SVM/RBF	Autism	autism_2024_Scores_RBF.xlsx	autism_2024_Scores_RBF_Top3.xlsx
XGBoost	Young vs Older	results/young_old_2024_Scores_XGBoost.xlsx	results/young_old_2024_Scores_XGBoost_Top3.xlsx
XGBoost	Autism	results/autism_2024_Scores_XGBoost.xlsx	results/autism_2024_Scores_XGBoost_Top3.xlsx

Important implementation detail

For XGBoost, the selected features were stored as feature indices in the Features column. These indices were converted back into feature names using the feature columns from the train/test files.

3. SVM/RBF SHAP Analysis

The SVM/RBF SHAP analysis was reviewed first because it was the existing reference pipeline. The Top 3 SVM models were used to identify the most important features in the best SVM/RBF models.

- For Young vs Older, the label mapping is Young = 0 and Older = 1.
- For Autism, the label mapping is Control = 0 and Autism = 1.
- The SHAP bar plots show mean absolute SHAP values, so they represent feature importance magnitude rather than direction.
- The top SVM features were then used to identify features containing _time.

The purpose of this step was to know which time variables should be checked in the raw data and in scatter plots.

4. XGBoost SHAP Implementation

After the SVM SHAP review, the same type of SHAP analysis was implemented for XGBoost. The initial code was large, so it was separated into a clean pipeline file and a reusable model utility file.

File	Role
pipelines_xgboost/phase4_4_SHAP_XGBoost.py	Small runner file. It reads config.yaml and calls the XGBoost SHAP function.
src/gaitml/models/get_SHAP_data_XGBoost.py	Main SHAP logic. It reads the Top 3 file, trains XGBoost models, calculates SHAP values, creates Excel outputs, and saves SHAP summary plots.

This structure made the pipeline easier to maintain and closer to the style of the existing SVM pipeline.

4.2 XGBoost SHAP Outputs

[XGBoost](#)

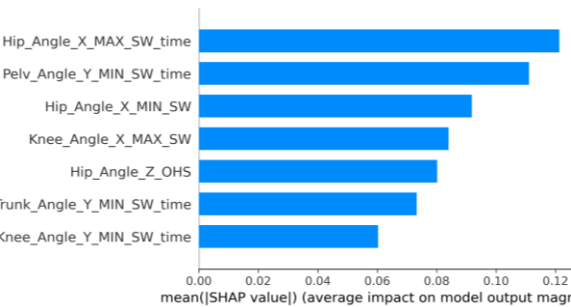
Dataset	Top 3 XGBoost models	SHAP Excel output	SHAP plot examples
Young vs Older	10833, 10844, 10823	output-young_old/young_old_2024_SHAP_data_XGBoost.xlsx	SHAP_XGBoost_10823_summary.png, SHAP_XGBoost_10833_summary.png, SHAP_XGBoost_10844_summary.png
Autism	4021, 3985, 8542	output-autism/autism_2024_SHAP_data_XGBoost.xlsx	SHAP_XGBoost_3985_summary.png, SHAP_XGBoost_4021_summary.png, SHAP_XGBoost_8542_summary.png

output-young_old

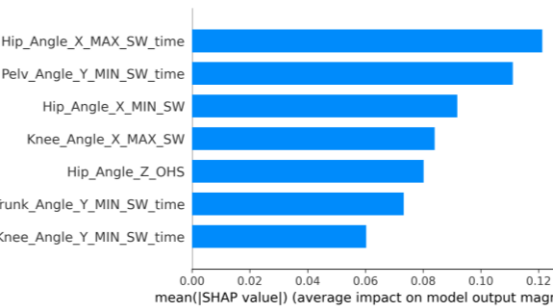
RESULTS: SVM

FOR YOUNG /OLD :

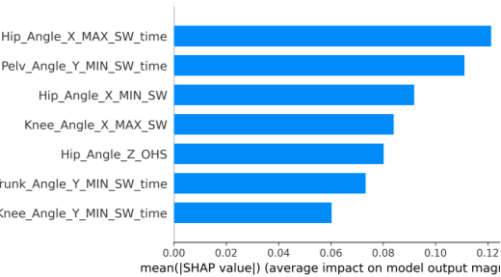
Name_Model:7603



Name_Model:7610



Name_Model:7617

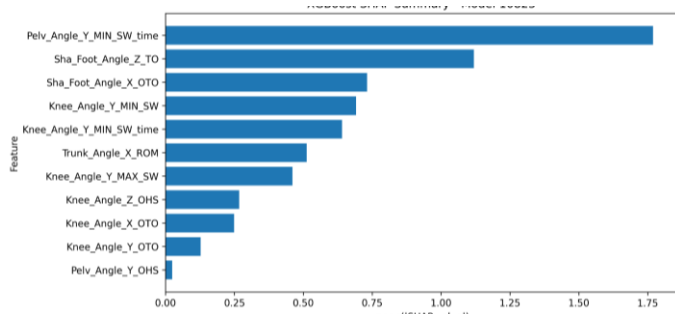


output-young_old

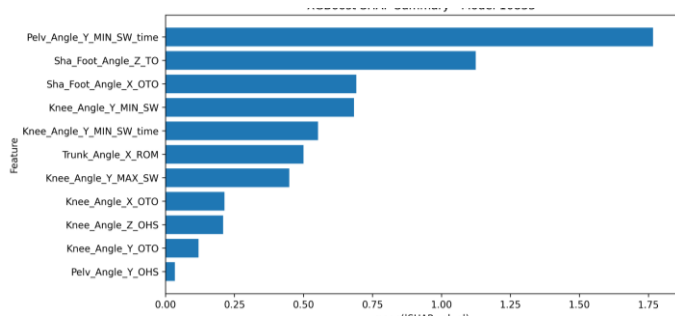
RESULTS: XGboost

FOR YOUNG /OLD :

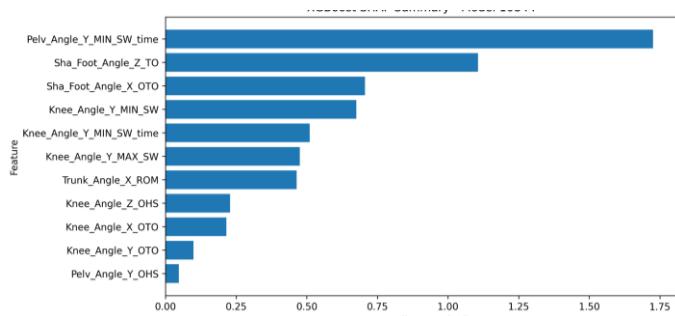
Name_Model:823



Name_Model:833



Name_Model:844



5. MLflow Logging

I also added MLflow tracking for the SVM and XGBoost results.

For Autism, the following files were logged successfully:

- configs/config.yaml
- results/autism_2024_JA_TS_features.xlsx
- results/autism_2024_NH_correlated_features.xlsx
- results/autism_2024_train.xlsx
- results/autism_2024_test.xlsx
- results/autism_2024_phase4_1_SFS_results_XGBoost.xlsx
- results/autism_2024_Step_2_Results_XGBoost.xlsx
- results/autism_2024_Scores_XGBoost.xlsx
- results/autism_2024_Scores_XGBoost_Top3.xlsx
- output-autism/autism_2024_SHAP_data_XGBoost.xlsx

6. SVM Time-Variable Raw Data and Statistics Check

Dr. Vicky asked to check the raw data and statistics on the time variables for the top features. To do this, the time-related features were extracted from the SVM Top 3 model files, and the raw processed train/test data was used to calculate descriptive statistics and create scatter plots.

Dataset	SVM time-related top features
Young vs Older	Hip_Angle_X_MAX_SW_time; Knee_Angle_Y_MIN_SW_time; Pelv_Angle_Y_MIN_SW_time; Trunk_Angle_Y_MIN_SW_time
Autism	Double_Limb_Support_Time; Knee_Angle_X_MAX_SW_time; Knee_Angle_Y_MAX_SW_time; Pelv_Angle_Y_MAX_SW_time; Trunk_Angle_Y_MAX_SW_time; Trunk_Angle_Z_MIN_SW_time

For each feature, the following statistics were calculated by group:

- Count
- Mean
- Standard deviation
- Median
- Minimum
- Maximum
- Q1
- Q3
- IQR

Output file	Content
young_old_2024_SVM_time_variable_stats.xlsx	Raw values, descriptive statistics, top time features, model-feature mapping, and potential outliers.
autism_2024_SVM_time_variable_stats.xlsx	Raw values, descriptive statistics, top time features, model-feature mapping, and potential outliers.

7. SVM Scatter Plots and Interpretation

Scatter plots were created because statistics alone give numbers, but scatter plots visually show whether the raw values look reasonable. In each scatter plot:

- X-axis = group
- Y-axis = raw value for the time feature
- Each dot = one sample
- Horizontal line = group mean

The scatter plots helped check whether the groups show visible differences, whether they overlap, and whether a few extreme values might be influencing the model. They are saved in these folders:

I created them to validate the raw values behind the top time-related features. The scatter plots help show whether the feature values look reasonable, whether the groups overlap, and whether outliers or missing values may affect the model.

[young_old_2024_SVM_time_variable_plots](#)

[autism_2024_SVM_time_variable_plots](#)

SVM Young/Old scatter plots: [young_old_2024_SVM_time_variable_plots/](#)

SVM Autism scatter plots: [autism_2024_SVM_time_variable_plots/](#)

Stats files: [young_old_2024_SVM_time_variable_stats.xlsx](#)

and [autism_2024_SVM_time_variable_stats.xlsx](#)

8. XGBoost Time-Variable Raw Data and Statistics Check

The same raw-data validation was then performed for XGBoost. This was important because Dr. Vicky asked why the XGBoost model had different features compared to SVM.

The XGBoost Top 3 files were used to extract time-related features. Compared to SVM, XGBoost selected fewer time-related features. In the Young vs Older XGBoost results, only a small number of time features were found, such as Pelv_Angle_Y_MIN_SW_time and Knee_Angle_Y_MIN_SW_time. This suggests that XGBoost may rely more on other biomechanical variables and less on time variables overall.

Output file	Purpose
young_old_2024_XGBoost_time_variable_stats.xlsx	Statistics and raw-data check for XGBoost time-related features in Young vs Older.
autism_2024_XGBoost_time_variable_stats.xlsx	Statistics and raw-data check for XGBoost time-related features in Autism.

The XGBoost scatter plots were created using the same approach as SVM, so the results could be compared visually between both model types.

9. Why XGBoost and SVM Selected Different Features

Dr. Vicky asked for a short summary explaining why the XGBoost model selected different features compared to SVM. The main interpretation is that the two algorithms learn from the data in different ways.

Model	How it learns	Why selected features can differ
SVM/RBF	Tries to find one smooth nonlinear boundary between the groups.	It selects features that help create the overall smooth separation.

XGBoost	Uses many decision trees and cutoff points.	It can select features that create useful thresholds or local split patterns.
---------	---	---

Therefore, even if SVM and XGBoost use the same dataset, they do not necessarily select the same features. XGBoost can focus on features that create strong decision splits, while SVM can focus on features that support the global boundary. Also, feature selection was run separately for each model type, so each model selected the best subset for its own learning method.

From the scatter plots, some XGBoost time features showed visible group-level differences, especially in the Older group. However, the groups still overlap, so these features should be interpreted as contributing features rather than features that separate the groups by themselves.

Jun 9, 2026

Today, I continued the MLflow documentation and experiment tracking work by improving the structure of the MLflow logging scripts for both datasets. The main objective was to organize the SVM and XGBoost results inside the same MLflow experiment while keeping them separated as different runs. This makes it easier to compare the two model types without mixing their artifacts, parameters, and metrics.

In addition to the MLflow integration work, I also reviewed all the comments and feedback provided by the AI team in this shared documentation. The goal was to make the documentation clearer, more complete, and easier to follow. Based on their comments, I clarified several parts of the workflow, especially the difference between Phase 1 feature filtering and Phase 2 correlation-based feature removal, the location of the generated files, the Top 3 model results, and the links or screenshots related to the outputs. This helped improve the documentation so that the reasoning, steps, results, and artifacts are easier for the team to review.

MLflow Organization:

The MLflow structure was organized by dataset. Each dataset has its own MLflow experiment, and inside each experiment, SVM and XGBoost are logged as two separate runs. This allows both models to be compared within the same dataset while keeping their files, metrics, parameters, and artifacts clearly separated.

For the Autism dataset, the MLflow experiment is named `Autism`. The corresponding logging script is `mlflow_autism.py`. This script creates two separate runs: one for the SVM RBF model and one for the XGBoost model.

For the Young vs Older dataset, the MLflow experiment is named `Young_and_Old`. The corresponding logging script is `mlflow_log_young_old.py`. This script also creates two separate runs: one for the SVM RBF model and one for the XGBoost model.

This structure makes the comparison clearer because the SVM and XGBoost results are stored under the same dataset experiment but separated by model type.

. Path Management

One important point was that the SVM and XGBoost files were not stored in the same project folder. The SVM files were located in the original project folder, while the XGBoost files were located in a separate modified project folder.

To solve this, two root paths were used in the MLflow scripts:

```
SVM_ROOT = Path(".")
```

and:

```
XGB_ROOT = Path(  
    r"C:\Users\k6qqj\Desktop\Gait-ML-Shap-main_y_Mohamed\Gait-ML-Shap-main"  
)
```

This allowed the script to collect SVM files from the current project folder and XGBoost files from the separate XGBoost project folder. This was necessary because using only relative paths caused some XGBoost files to be skipped or not found.

. Logged Information

For each MLflow run, the script logs the main files and outputs related to the corresponding model. The goal was to make sure that all important results can be accessed directly from MLflow.

The logged information includes:

- Top 3 model results
- CV accuracy
- Test accuracy
- Sensitivity
- Specificity
- Selected features
- Model hyperparameters
- Train and test files
- SHAP Excel files
- SHAP summary plots
- Time-variable statistics files
- Time-variable scatter plots

For SVM, the logged artifacts include the official SVM RBF outputs, such as the Top 3 SVM results, SHAP files, and time-variable validation outputs.

For XGBoost, the logged artifacts include the experimental XGBoost outputs, such as the Top 3 XGBoost results, SHAP files, and XGBoost time-variable validation outputs.

I also started setting up a tunnel server to share the MLflow dashboard with the team in real time. The goal is to allow other team members to access the MLflow interface from their own computers instead of only viewing it locally on my laptop. However, this part is still not fully completed yet because the public link depends on the local MLflow server, the tunnel session, and the laptop remaining active.

Jun 10, 2026– Jun 12, 2026

I helped Ben with the V3D work. Most of the day was focused on verifying the diabetics dataset and checking that the data was correct before continuing with the analysis.

Jun 15, 2026

Objective

Today, the main objective was to continue the machine learning model comparison work by adding a Random Forest-based pipeline and integrating the results into MLflow. The goal was to compare SVM RBF, XGBoost, and Random Forest on the Autism dataset using the same general workflow, including feature selection, model evaluation, SHAP interpretation, and experiment tracking.

1. Random Forest Pipeline Work

Today, I worked on adapting the existing machine learning pipeline to support Random Forest as an additional classification model. The goal was to follow the same structure already used for SVM and XGBoost so that the results can be compared fairly.

The Random Forest pipeline was organized under:

`pipelines_Random_forest`

The Random Forest workflow follows the same phases as the other models:

Phase 4.1: Sequential Feature Selection

Phase 4.2: Model combinations and hyperparameter testing

Phase 4.3: Model scoring and ranking

Phase 4.4: SHAP analysis

This allows Random Forest to be evaluated using the same Autism dataset, the same train/test split, and similar evaluation metrics.

2. Random Forest Configuration

I prepared the configuration for Random Forest in the `config.yaml` file. The configuration defines the input and output files used by the Random Forest pipeline.

The main Random Forest output files are:

`results/autism_2024_phase4_1_SFS_results_RandomForest.xlsx`

`results/autism_2024_Step_2_Results_RandomForest.xlsx`

`results/autism_2024_Scores_RandomForest.xlsx`

`results/autism_2024_Scores_RandomForest_Top3.xlsx`

`output-autism/autism_2024_SHAP_data_RandomForest.xlsx`

`output-autism/autism_2024_SHAP_RandomForest`

These files store the SFS results, model combination results, final scores, top 3 selected models, SHAP Excel output, and SHAP plots.

3. Random Forest Model Combination and Scoring

I worked on the Random Forest model combination phase to test different Random Forest hyperparameters. The goal was to evaluate multiple configurations and identify the best-performing models.

The tested Random Forest parameters included:

n_estimators,max_depth,min_samples_split,min_samples_leaf,max_features

The results were ranked using model performance metrics such as:

CV accuracy,Test accuracy,Sensitivity,Specificity,F1 score,MCC

A Top 3 Random Forest results file was then used for tracking and comparison.

4. Random Forest SHAP Analysis

I also worked on the Random Forest SHAP part. The objective was to generate SHAP explanations for the selected top Random Forest models.

The SHAP output includes:

SHAP Excel file

SHAP summary plots

Feature importance values

Selected feature indices

Selected feature names

This helps explain which biomechanical features contributed the most to the Random Forest model predictions.

5. MLflow Integration

After preparing the Random Forest outputs, I worked on integrating all model results into MLflow.

The goal was to track the Autism model comparison between:

SVM RBF

XGBoost

Random Forest

The MLflow script logs:

Top 3 model metrics

CV accuracy

Test accuracy

Sensitivity

Specificity

F1 score

MCC

Test class counts
Excel output files
SHAP plots
Related artifacts

The final MLflow script was added as:

`pipelines/mlflow_log_autism.py`

This script allows the Autism model comparison results to be reviewed in one MLflow experiment.

6. MLflow Local and Shared Server Discussion

MLflow was working locally on my machine. I asked Ashirbad and Karansinh how we could make MLflow accessible to everyone instead of only running it locally.

Ashirbad suggested checking DagsHub as a possible option, but he also mentioned that one issue is that it may be an open/public server. This means we need to be careful about privacy and data sharing before using it.

7. GitHub PR Preparation

Ashirbad asked me to create a Pull Request for the MLflow integration and to keep the PR small with small commits.

At first, the PR contained too many changes, including several unrelated modified files. To fix this, I cleaned the Git branch and kept only the MLflow integration file.

The cleanup process included:

- Saving unrelated local changes using `git stash`
- Resetting the branch to main
- Cherry-picking only the MLflow commit
- Force-pushing the cleaned branch
- Creating a new clean Pull Request

8. Final Pull Request Status

A new clean Pull Request was created for the MLflow integration.

The PR title is:

Add MLflow logging for Autism model comparison

The final PR contains:

1 commit

1 file changed

The changed file is:

`pipelines/mlflow_log_autism.py`

The PR has no conflicts with the main branch and is ready for review. I did not merge it myself because it should first be reviewed by Ashirbad and Karansinh.

9. Summary

Today, I completed important work on the Random Forest pipeline, including configuration, model evaluation, Top 3 results, and SHAP analysis. I also integrated SVM RBF, XGBoost, and Random Forest results into MLflow for the Autism dataset. Finally, I created a clean and focused GitHub Pull Request containing only the MLflow integration file, as requested by Ashirbad.

Jun 16, 2026

-MLflow/DagsHub Integration

-Flatfoot Pipeline Setup

Today, I continued the MLflow integration work and moved from local-only experiment tracking to a remote tracking setup using DagsHub. First, I tested the MLflow connection with DagsHub

using a temporary repository under my account. After solving authentication and repository URL issues, the remote MLflow tracking started working correctly. I confirmed that experiment runs can be created and viewed from DagsHub, which means the setup can be accessed from other machines as long as users have the correct repository access and their own DagsHub token.

I updated both `mlflow_log_autism.py` and `mlflow_log_young_old.py` so they now support two modes: local MLflow tracking using SQLite and remote MLflow tracking using DagsHub. The scripts now read the tracking URI, experiment name, and credentials from environment variables instead of hardcoding them. I also added safer artifact controls. By default, raw datasets and train/test files are not uploaded to DagsHub for privacy reasons. When needed, result artifacts such as output files and SHAP plots can be enabled while still keeping raw data disabled.

Here is the link : <https://dagshub.com/aminejemni181/Gait-ML-Shap-Test.mlflow>

After that, I started preparing the new Control vs Flatfoot dataset, using the file `ALL FEATURES OLDER - Pes Planus and Control JUNE 25.xlsx`. The pipeline configuration was adapted for this dataset with the correct paths, output names, and label mapping: `Control = 0` and `Flatfoot/Pes Planus = 1`.

The Flatfoot pipeline was prepared and tested for both **SVM** and **XGBoost**. For each pipeline, we followed the same full workflow: Phase 1 feature extraction, Phase 2 correlated-feature removal, Phase 3 train/test split, Phase 4.1 SFS, Phase 4.2 model combinations, score generation, Top 3 model selection, and SHAP output preparation.

Random Forest is still the remaining part to complete for the Flatfoot dataset. Once it is added, we will have the same complete comparison structure as the previous datasets: SVM, XGBoost, and Random Forest.

Jun 17, 2026

Objective

Today, I worked on improving the experiment tracking setup for the Flatfoot dataset using MLflow and DagsHub. The main goal was to make the MLflow results accessible outside my local machine, so the team can later review the experiments more easily.

Work Completed

I started by working on the MLflow integration for the Flatfoot dataset. The objective was to track the machine learning results in a structured way, similar to what was done for the Autism and Young/Old datasets.

The MLflow tracking includes model parameters, evaluation metrics, output files, and artifacts generated by the pipeline. This helps organize the results and makes it easier to compare different runs.

Since the local MLflow setup works only on my machine, I also explored using DagsHub as a remote MLflow tracking platform. The purpose of DagsHub is to allow MLflow experiments to be stored and viewed through an online interface instead of being limited to a local MLflow UI.

DagsHub Setup

I configured MLflow to connect with DagsHub as a remote tracking server. This allows experiment runs to be logged directly to DagsHub instead of only being saved in a local `mlflow.db` file.

The goal of this setup is to make the Flatfoot experiment results accessible from a shared platform, where supervisors and team members can view the logged runs, metrics, parameters, and artifacts.

This is important because local MLflow is useful for testing, but it is not practical for team collaboration. DagsHub provides a possible solution for sharing the MLflow dashboard.

Discussion with the Team

I asked Ashirbad and Karansinh how to make MLflow work beyond my local machine. Ashirbad suggested checking DagsHub as a possible solution, but he also mentioned that one issue is that DagsHub may be an open/public server.

Because of this, we need to be careful about privacy and data sharing before using DagsHub for sensitive data. The next step is to confirm whether DagsHub is acceptable for this project or whether another shared server setup should be used.

Summary

Today, I worked on connecting the Flatfoot MLflow tracking workflow with DagsHub. This is a first step toward making the experiment results accessible to the team instead of keeping them only on my local machine. The setup still needs to be reviewed, especially regarding privacy and whether DagsHub is the best option for shared MLflow tracking.

Daily ML Pipeline Updates: Jun 18, 2026 - Jun 30, 2026

This section replaces the short global summary with a daily work log. Each day is written across two pages so the work from ALL_MODELS is easier to review day by day.

Jun 18, 2026

Flatfoot and SVM Reference Consolidation - Page 1 of 2

I started this period by continuing the Flatfoot work from the previous days and using the SVM pipeline as the reference baseline. The first objective was to make sure the Flatfoot Control vs Pes Planus dataset followed the same logic already used for Autism and Young vs Older.

I reviewed the dataset naming and kept the label mapping clear: Control = 0 and Flatfoot / Pes Planus = 1. This was important because every later model comparison depends on using the same class definitions and the same train/test split logic.

I also checked that the SVM outputs could be used as the reference results when comparing additional model families. The SVM folder became the baseline area for reviewing outputs, Top 3 files, MLflow logs, and time-variable statistics.

Work Completed

Checked the Flatfoot dataset setup and naming convention.

- Checked the Flatfoot dataset setup and naming convention.

Kept SVM/RBF as the reference baseline for later comparisons.

- Kept SVM/RBF as the reference baseline for later comparisons.

Reviewed train/test, score, Top 3, and SHAP output naming patterns.

- Reviewed train/test, score, Top 3, and SHAP output naming patterns.

Prepared the work so the same dataset structure could be reused by other models.

- Prepared the work so the same dataset structure could be reused by other models.

Files / Areas Reviewed

Item	Details
Dataset	Flatfoot Control vs Pes Planus
Reference model	SVM/RBF
Main focus	Prepare consistent baseline outputs and naming
Next step	Start expanding the same workflow to additional models

Jun 18, 2026

Flatfoot and SVM Reference Consolidation - Page 2 of 2

The main result of this day was not only one new file, but a cleaner understanding of how the remaining model work should be organized. I confirmed that the project needed a consistent ALL_MODELS structure instead of mixing all experiments inside one folder.

This helped avoid overwriting outputs between SVM, XGBoost, Random Forest, and the new models. It also made it easier to explain the project in the documentation, because each model family could have its own folder and its own phase files.

My Understanding

My understanding is that this work is part of the transition from a single-pipeline report into a complete model-comparison workspace. The important point is to keep the same datasets, preprocessing logic, and output naming structure, so the final interpretation can focus on how the models behave differently.

Result / Next Step

Keep the model-specific folder structure clear and avoid mixing outputs between models.

- Keep the model-specific folder structure clear and avoid mixing outputs between models.

Use the same dataset names and model suffixes in result files.

- Use the same dataset names and model suffixes in result files.

Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

- Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

Respect the existing documentation already stored inside the project folders.

- Respect the existing documentation already stored inside the project folders.

Jun 19, 2026

XGBoost Review and Cross-Model Naming - Page 1 of 2

I continued by reviewing the existing XGBoost work and checking how its files were named compared with the SVM files. The goal was to make sure XGBoost could stay as a separate comparison model without damaging the original SVM pipeline.

I checked the XGBoost outputs such as Step 2 result files, score files, Top 3 files, and SHAP data files. I also kept the same dataset naming pattern: autism_2024, young_old_2024, and flatfoot_control_older_2024.

This day was important because XGBoost already had a different model logic from SVM, so it gave a good template for how to add more non-SVM models later.

Work Completed

Reviewed the existing XGBoost project folder.

- Reviewed the existing XGBoost project folder.

Confirmed that XGBoost should stay separate from the SVM reference folder.

- Confirmed that XGBoost should stay separate from the SVM reference folder.

Kept output names consistent across Autism, Young vs Older, and Flatfoot.

- Kept output names consistent across Autism, Young vs Older, and Flatfoot.

Used XGBoost as the first example for multi-model comparison organization.

- Used XGBoost as the first example for multi-model comparison organization.

Files / Areas Reviewed

Item	Details
Model	XGBoost
Folder	Gait-ML-Shap-main_y_Mohamed_XGboost
Main work	Review outputs and naming
Documentation point	Use model suffixes in result filenames

Jun 19, 2026

XGBoost Review and Cross-Model Naming - Page 2 of 2

I also considered how the score and SHAP outputs should be logged later in MLflow. Since each model has its own selected features and its own interpretation files, the model name must be present in the output filename.

This reduces confusion when the team reviews files such as Scores_XGBoost, Scores_RBF, or SHAP_data_XGBoost. The same rule was then reused for MLP, LightGBM, CatBoost, FT-Transformer, and Autoencoder Classifier.

My Understanding

My understanding is that this work is part of the transition from a single-pipeline report into a complete model-comparison workspace. The important point is to keep the same datasets, preprocessing logic, and output naming structure, so the final interpretation can focus on how the models behave differently.

Result / Next Step

Keep the model-specific folder structure clear and avoid mixing outputs between models.

- Keep the model-specific folder structure clear and avoid mixing outputs between models.

Use the same dataset names and model suffixes in result files.

- Use the same dataset names and model suffixes in result files.

Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

- Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

Respect the existing documentation already stored inside the project folders.

- Respect the existing documentation already stored inside the project folders.

Jun 20, 2026

Planning the ALL_MODELS Expansion - Page 1 of 2

I planned the larger ALL_MODELS structure so that every model could be handled in a separate workspace. The goal was to avoid mixing code and results between models while still keeping the same ML pipeline phases.

The model families selected for the expansion were MLP, LightGBM, CatBoost, FT-Transformer, and Autoencoder Classifier. These models were added because they provide different learning approaches and can be compared with SVM, XGBoost, and Random Forest.

I decided to keep one folder per model family. This makes the work safer because a change in one model pipeline does not directly overwrite another model pipeline.

Work Completed

Prepared the ALL_MODELS folder strategy.

- Prepared the ALL_MODELS folder strategy.

Planned one separate folder per model family.

- Planned one separate folder per model family.

Kept the same pipeline phase order for all models.

- Kept the same pipeline phase order for all models.

Prepared the naming convention for model-specific outputs.

- Prepared the naming convention for model-specific outputs.

Files / Areas Reviewed

Item	Details
Folder strategy	One folder per model
Models added	MLP, LightGBM, CatBoost, FT-Transformer, Autoencoder Classifier
Main reason	Avoid mixing outputs and code
Comparison goal	Keep preprocessing consistent across models

Jun 20, 2026

Planning the ALL_MODELS Expansion - Page 2 of 2

The main pipeline phases stayed the same: Phase 1 feature review, Phase 2 correlated-feature removal, Phase 3 train/test split, Phase 4.1 feature selection or feature preparation, Phase 4.2 model training or combinations, Phase 4.3 scoring, Top 3 selection, and Phase 4.4 SHAP analysis.

By keeping this same structure, the final comparison can focus on model behavior and performance instead of being confused by different preprocessing steps.

My Understanding

My understanding is that this work is part of the transition from a single-pipeline report into a complete model-comparison workspace. The important point is to keep the same datasets, preprocessing logic, and output naming structure, so the final interpretation can focus on how the models behave differently.

Result / Next Step

Keep the model-specific folder structure clear and avoid mixing outputs between models.

- Keep the model-specific folder structure clear and avoid mixing outputs between models.

Use the same dataset names and model suffixes in result files.

- Use the same dataset names and model suffixes in result files.

Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

- Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

Respect the existing documentation already stored inside the project folders.

- Respect the existing documentation already stored inside the project folders.

Jun 21, 2026

MLP Pipeline Preparation - Page 1 of 2

I worked on preparing the MLP pipeline as one of the additional model families. The MLP folder follows the same general pipeline structure as the earlier models, but the model training and scoring logic is adapted for a neural-network classifier.

The MLP outputs were organized for Autism, Young vs Older, and Flatfoot. This included JA/TS feature files, non-highly-correlated feature files, train/test files, SFS results, Step 2 model results, score files, Top 3 score files, and SHAP data files.

The goal was to make MLP comparable with the existing SVM, XGBoost, and Random Forest work.

Work Completed

Created or organized the pipelines_MLP folder.

- Created or organized the pipelines_MLP folder.

Prepared Phase 4.1, Phase 4.2, Phase 4.3, and Phase 4.4 MLP scripts.

- Prepared Phase 4.1, Phase 4.2, Phase 4.3, and Phase 4.4 MLP scripts.

Generated or organized MLP result files for the three datasets.

- Generated or organized MLP result files for the three datasets.

Kept MLP output names consistent with the rest of the project.

- Kept MLP output names consistent with the rest of the project.

Files / Areas Reviewed

Item	Details
Model	MLP
Pipeline folder	pipelines_MLP
Datasets	Autism, Young vs Older, Flatfoot
Key outputs	SFS, Step 2, Scores, Top3, SHAP data

Jun 21, 2026

MLP Pipeline Preparation - Page 2 of 2

The MLP work is important because it tests whether a neural-network model can learn useful patterns from the gait features. Since MLP can model nonlinear relationships, it gives another comparison point against SVM/RBF and tree-based models.

I also kept the result files readable by adding the MLP suffix in the filenames, such as Scores_MLP, Scores_MLP_Top3, Step_2_Results_MLP, and SHAP_data_MLP.

My Understanding

My understanding is that this work is part of the transition from a single-pipeline report into a complete model-comparison workspace. The important point is to keep the same datasets, preprocessing logic, and output naming structure, so the final interpretation can focus on how the models behave differently.

Result / Next Step

Keep the model-specific folder structure clear and avoid mixing outputs between models.

- Keep the model-specific folder structure clear and avoid mixing outputs between models.

Use the same dataset names and model suffixes in result files.

- Use the same dataset names and model suffixes in result files.

Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

- Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

Respect the existing documentation already stored inside the project folders.

- Respect the existing documentation already stored inside the project folders.

Jun 22, 2026

LightGBM Pipeline Preparation - Page 1 of 2

I continued the all-model expansion by preparing the LightGBM pipeline. LightGBM is a gradient boosting model and gives another tree-based comparison model beside XGBoost and CatBoost.

I organized LightGBM phase files for feature preparation, model combinations, scoring, and SHAP analysis. The output files were prepared using the same dataset prefixes so the results stay comparable with the other model folders.

The LightGBM work included result files for Autism, Young vs Older, and Flatfoot, with score and Top 3 outputs saved using the LightGBM suffix.

Work Completed

Prepared pipelines_LightGBM.

- Prepared pipelines_LightGBM.

Added LightGBM-specific model-combination and scoring scripts.

- Added LightGBM-specific model-combination and scoring scripts.

Organized Scores_LightGBM and Scores_LightGBM_Top3 files.

- Organized Scores_LightGBM and Scores_LightGBM_Top3 files.

Prepared SHAP data outputs for LightGBM.

- Prepared SHAP data outputs for LightGBM.

Files / Areas Reviewed

Item	Details
Model	LightGBM
Pipeline folder	pipelines_LightGBM
Main comparison	Tree boosting model compared with XGBoost and CatBoost
Key outputs	Scores_LightGBM, Top3, SHAP data

Jun 22, 2026

LightGBM Pipeline Preparation - Page 2 of 2

The reason for adding LightGBM is that it can handle tabular features efficiently and may perform differently from XGBoost, even though both are boosting methods. This gives the project a stronger model comparison instead of relying on one tree-based algorithm only.

I also kept the folder documentation and project docs untouched. The existing extras/docs files stayed in place because they are part of the project structure and should not be overwritten by the progress report.

My Understanding

My understanding is that this work is part of the transition from a single-pipeline report into a complete model-comparison workspace. The important point is to keep the same datasets, preprocessing logic, and output naming structure, so the final interpretation can focus on how the models behave differently.

Result / Next Step

Keep the model-specific folder structure clear and avoid mixing outputs between models.

- Keep the model-specific folder structure clear and avoid mixing outputs between models.

Use the same dataset names and model suffixes in result files.

- Use the same dataset names and model suffixes in result files.

Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

- Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

Respect the existing documentation already stored inside the project folders.

- Respect the existing documentation already stored inside the project folders.

Jun 23, 2026

CatBoost Pipeline Preparation - Page 1 of 2

I prepared the CatBoost pipeline as another boosting-based model. The CatBoost folder includes phase scripts, runner files, shared pipeline logic, top-feature scripts, score generation, and SHAP output preparation.

The main objective was to keep CatBoost organized in the same way as the other model folders while allowing CatBoost-specific training and evaluation logic. This makes it possible to compare CatBoost with XGBoost, LightGBM, Random Forest, SVM, and MLP.

The output files were prepared for Autism, Young vs Older, and Flatfoot using consistent names such as Scores_CatBoost, Scores_CatBoost_Top3, Step_2_Results_CatBoost, and SHAP_data_CatBoost.

Work Completed

Prepared pipelines_CATBOOST.

- Prepared pipelines_CATBOOST.

Added CatBoost runner and common pipeline files.

- Added CatBoost runner and common pipeline files.

Prepared CatBoost result files for the three datasets.

- Prepared CatBoost result files for the three datasets.

Kept Top 3 and SHAP outputs model-specific.

- Kept Top 3 and SHAP outputs model-specific.

Files / Areas Reviewed

Item	Details
Model	CatBoost
Pipeline folder	pipelines_CATBOOST
Additional scripts	run_catboost_pipeline.py, pipeline_common.py, top-feature scripts
Key outputs	Step 2, Scores, Top3, SHAP data

Jun 23, 2026

CatBoost Pipeline Preparation - Page 2 of 2

CatBoost is useful because it is another strong model for tabular data. Even if the dataset does not contain many categorical variables, CatBoost can still provide a robust boosting comparison.

I also kept top-feature scripts for Autism and Young vs Older so the selected features can be reviewed after model scoring. This is important because the final interpretation should include both performance metrics and the features selected by each model.

My Understanding

My understanding is that this work is part of the transition from a single-pipeline report into a complete model-comparison workspace. The important point is to keep the same datasets, preprocessing logic, and output naming structure, so the final interpretation can focus on how the models behave differently.

Result / Next Step

Keep the model-specific folder structure clear and avoid mixing outputs between models.

- Keep the model-specific folder structure clear and avoid mixing outputs between models.

Use the same dataset names and model suffixes in result files.

- Use the same dataset names and model suffixes in result files.

Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

- Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

Respect the existing documentation already stored inside the project folders.

- Respect the existing documentation already stored inside the project folders.

Jun 24, 2026

FT-Transformer Pipeline Preparation - Page 1 of 2

I worked on the FT-Transformer pipeline to add a transformer-based tabular model to the comparison. This model is different from SVM, tree-based models, and MLP because it uses a transformer-style architecture for tabular features.

The FT-Transformer folder includes phase scripts for feature preparation, model training, score generation, and SHAP output preparation. I kept the same dataset structure and output naming convention used by the rest of ALL_MODELS.

The purpose was to test whether a deeper tabular model could provide useful performance or feature behavior compared with the traditional ML models.

Work Completed

Prepared pipelines_FTTTransformer.

- Prepared pipelines_FTTTransformer.

Organized FT-Transformer result files for Autism, Young vs Older, and Flatfoot.

- Organized FT-Transformer result files for Autism, Young vs Older, and Flatfoot.

Prepared Scores_FTTTransformer and Top3 outputs.

- Prepared Scores_FTTTransformer and Top3 outputs.

Prepared SHAP data outputs for FT-Transformer.

- Prepared SHAP data outputs for FT-Transformer.

Files / Areas Reviewed

Item	Details
Model	FT-Transformer
Pipeline folder	pipelines_FTTTransformer
Model type	Transformer-style tabular model
Key outputs	SFS, Step 2, Scores, Top3, SHAP data

Jun 24, 2026

FT-Transformer Pipeline Preparation - Page 2 of 2

This day also required checking that the folder structure did not conflict with the Autoencoder Classifier work, because both are deep-learning-style experiments. I kept FT-Transformer as its own folder so it can be reviewed independently.

The FT-Transformer outputs follow the same naming pattern: phase4_1_SFS_results_FTTTransformer, Step_2_Results_FTTTransformer, Scores_FTTTransformer, Scores_FTTTransformer_Top3, and SHAP_data_FTTTransformer.

My Understanding

My understanding is that this work is part of the transition from a single-pipeline report into a complete model-comparison workspace. The important point is to keep the same datasets, preprocessing logic, and output naming structure, so the final interpretation can focus on how the models behave differently.

Result / Next Step

Keep the model-specific folder structure clear and avoid mixing outputs between models.

- Keep the model-specific folder structure clear and avoid mixing outputs between models.

Use the same dataset names and model suffixes in result files.

- Use the same dataset names and model suffixes in result files.

Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

- Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

Respect the existing documentation already stored inside the project folders.

- Respect the existing documentation already stored inside the project folders.

Jun 25, 2026

Autoencoder Classifier Structure - Page 1 of 2

I prepared the Autoencoder Classifier structure as another experimental model family. This model is different because it can learn a compressed representation of the gait features before classification.

The autoencoder_classifier folder was organized with its own configs, data folders, source files, and pipelines_AutoencoderClassifier phase scripts. This keeps it separate from the FT-Transformer and MLP folders.

The goal was to prepare the project so an autoencoder-based classifier can later be tested using the same Autism, Young vs Older, and Flatfoot dataset structure.

Work Completed

Created or organized autoencoder_classifier.

- Created or organized autoencoder_classifier.

Prepared pipelines_AutoencoderClassifier phase files.

- Prepared pipelines_AutoencoderClassifier phase files.

Kept configs and data folders separate inside the autoencoder project.

- Kept configs and data folders separate inside the autoencoder project.

Prepared the folder for later representation-learning experiments.

- Prepared the folder for later representation-learning experiments.

Files / Areas Reviewed

Item	Details
Model	Autoencoder Classifier
Folder	autoencoder_classifier/pipelines_AutoencoderClassifier
Main purpose	Representation-learning classifier experiment
Status	Pipeline structure prepared

Jun 25, 2026

Autoencoder Classifier Structure - Page 2 of 2

This work helps expand the project beyond standard classifiers. If the autoencoder can learn useful low-dimensional gait representations, it may support a different interpretation of the data.

At this stage, the most important task was organization and preparation: making sure the folder has the expected pipeline scripts and follows the same workflow style as the rest of ALL_MODELS.

My Understanding

My understanding is that this work is part of the transition from a single-pipeline report into a complete model-comparison workspace. The important point is to keep the same datasets, preprocessing logic, and output naming structure, so the final interpretation can focus on how the models behave differently.

Result / Next Step

Keep the model-specific folder structure clear and avoid mixing outputs between models.

- Keep the model-specific folder structure clear and avoid mixing outputs between models.

Use the same dataset names and model suffixes in result files.

- Use the same dataset names and model suffixes in result files.

Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

- Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

Respect the existing documentation already stored inside the project folders.

- Respect the existing documentation already stored inside the project folders.

Jun 26, 2026

All-Model MLflow Logging - Page 1 of 2

I continued the MLflow organization work by preparing all-model logging scripts. The objective was to move from logging only one or two model types toward logging all model families under the correct dataset experiment.

In the SVM/reference area, I prepared all-model MLflow files for Autism, Young vs Older, and Flatfoot. These scripts are intended to collect results from multiple model folders and make them easier to compare in MLflow.

This step is important because the project now contains many model families, so the results need a tracking structure that is clearer than manually opening many Excel files.

Work Completed

Prepared mlflow_log_autism_all_models.py.

- Prepared mlflow_log_autism_all_models.py.

Prepared mlflow_log_young_old_all_models.py.

- Prepared mlflow_log_young_old_all_models.py.

Prepared mlflow_log_flatfoot_all_models.py.

- Prepared mlflow_log_flatfoot_all_models.py.

Organized model outputs for comparison by dataset.

- Organized model outputs for comparison by dataset.

Files / Areas Reviewed

Item	Details
Tracking tool	MLflow
Main scripts	mlflow_log_autism_all_models.py; mlflow_log_young_old_all_models.py; mlflow_log_flatfoot_all_models.py
Comparison level	Dataset first, then model type
Privacy note	Avoid uploading raw datasets unless approved

Jun 26, 2026

All-Model MLflow Logging - Page 2 of 2

The MLflow structure should separate runs by model type while keeping the dataset as the main comparison group. For example, Autism can include SVM, XGBoost, Random Forest, MLP, LightGBM, CatBoost, FT-Transformer, and Autoencoder runs.

The important point is that raw data privacy must still be respected. Result artifacts and summary outputs can be useful for review, but raw datasets should not be uploaded unless the team confirms it is allowed.

My Understanding

My understanding is that this work is part of the transition from a single-pipeline report into a complete model-comparison workspace. The important point is to keep the same datasets, preprocessing logic, and output naming structure, so the final interpretation can focus on how the models behave differently.

Result / Next Step

Keep the model-specific folder structure clear and avoid mixing outputs between models.

- Keep the model-specific folder structure clear and avoid mixing outputs between models.

Use the same dataset names and model suffixes in result files.

- Use the same dataset names and model suffixes in result files.

Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

- Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

Respect the existing documentation already stored inside the project folders.

- Respect the existing documentation already stored inside the project folders.

Jun 27, 2026

Backend API and Research Assistant Preparation - Page 1 of 2

I started preparing the backend service so model results can be accessed through an interface instead of only through folders and Excel files. The backend includes routes for models, datasets, comparison, and the research assistant.

The backend work includes main.py, routers, schemas, services, and an Agents folder. The services are responsible for loading data and figures, while the routers expose the information through API endpoints.

This is useful because the project is becoming too large to review manually from folders only. A backend layer makes it possible to connect the results to a dashboard.

Work Completed

Created backend API structure.

- Created backend API structure.

Added routers for models, datasets, comparison, and agent endpoints.

- Added routers for models, datasets, comparison, and agent endpoints.

Prepared services for data loading and figure handling.

- Prepared services for data loading and figure handling.

Added agent knowledge files about metrics, models, overview, and pipelines.

- Added agent knowledge files about metrics, models, overview, and pipelines.

Files / Areas Reviewed

Item	Details
Backend file area	Purpose
routers/models.py	Model endpoints
routers/datasets.py	Dataset endpoints
routers/comparison.py	Comparison endpoints
Agents/knowledge	Project explanation for assistant

Jun 27, 2026

Backend API and Research Assistant Preparation - Page 2 of 2

I also prepared the agent knowledge folder so the assistant can answer questions about the project using local documentation. The knowledge files include metrics, model descriptions, pipeline overview, and project context.

The goal is not only to show metrics, but also to help explain what the metrics mean and where the results come from. This can make the dashboard more useful for review meetings and documentation.

My Understanding

My understanding is that this work is part of the transition from a single-pipeline report into a complete model-comparison workspace. The important point is to keep the same datasets, preprocessing logic, and output naming structure, so the final interpretation can focus on how the models behave differently.

Result / Next Step

Keep the model-specific folder structure clear and avoid mixing outputs between models.

- Keep the model-specific folder structure clear and avoid mixing outputs between models.

Use the same dataset names and model suffixes in result files.

- Use the same dataset names and model suffixes in result files.

Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

- Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

Respect the existing documentation already stored inside the project folders.

- Respect the existing documentation already stored inside the project folders.

Jun 28, 2026

Frontend Dashboard and Model Review Interface - Page 1 of 2

I worked on the frontend dashboard using React and Vite. The goal was to create a visual interface where the model results can be reviewed more easily than opening many Excel files manually.

The frontend includes pages for Home, Datasets, Models Overview, Model Detail, Comparison, and Biomechanics. It also includes reusable components for metric cards, tables, charts, confusion matrices, feature lists, figure galleries, dataset selection, and the gait research assistant.

This makes the project easier to present because the results can be organized into views instead of only being stored as files.

Work Completed

Prepared React/Vite frontend structure.

- Prepared React/Vite frontend structure.

Added pages for datasets, models, details, comparison, and biomechanics.

- Added pages for datasets, models, details, comparison, and biomechanics.

Added reusable components for metrics, charts, features, figures, and assistant UI.

- Added reusable components for metrics, charts, features, figures, and assistant UI.

Connected frontend logic to backend API service structure.

- Connected frontend logic to backend API service structure.

Files / Areas Reviewed

Item	Details
Frontend area	Files / examples
Pages	Home, Datasets, ModelsOverview, ModelDetail, Comparison, Biomechanics
Components	MetricsTable, MetricCard, ComparisonChart, ConfusionMatrix, FeatureList, FigureGallery
Assets	Biomechanics screenshots, images, and media

Jun 28, 2026

Frontend Dashboard and Model Review Interface - Page 2 of 2

I also added biomechanics media assets such as screenshots, images, and video files. These assets support the Biomechanics page and help explain the gait-analysis context behind the machine learning work.

The frontend is an important step because it turns the project from only a pipeline/results folder into a review tool. It can help the team compare models, inspect datasets, and understand the meaning of the outputs.

My Understanding

My understanding is that this work is part of the transition from a single-pipeline report into a complete model-comparison workspace. The important point is to keep the same datasets, preprocessing logic, and output naming structure, so the final interpretation can focus on how the models behave differently.

Result / Next Step

Keep the model-specific folder structure clear and avoid mixing outputs between models.

- Keep the model-specific folder structure clear and avoid mixing outputs between models.

Use the same dataset names and model suffixes in result files.

- Use the same dataset names and model suffixes in result files.

Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

- Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

Respect the existing documentation already stored inside the project folders.

- Respect the existing documentation already stored inside the project folders.

Jun 29, 2026

Backend/Frontend Integration and Final ALL_MODELS Review - Page 1 of 2

I continued working on the backend and frontend integration. The goal was to make sure the dashboard structure can connect to the backend API and display useful model information.

I reviewed the ALL_MODELS folders again to make sure the important model families are present: SVM, XGBoost, Random Forest, MLP, LightGBM, CatBoost, FT-Transformer, Autoencoder Classifier, backend, and frontend.

This day focused on connecting the technical pipeline work with the interface work, so the project can be reviewed in a more organized way.

Work Completed

Reviewed backend routes and frontend pages.

- Reviewed backend routes and frontend pages.

Checked that model folders and result patterns were consistent.

- Checked that model folders and result patterns were consistent.

Connected the dashboard idea with the ALL_MODELS output organization.

- Connected the dashboard idea with the ALL_MODELS output organization.

Prepared the project for easier review by the team.

- Prepared the project for easier review by the team.

Files / Areas Reviewed

Item	Details
Main focus	Backend/frontend integration and ALL_MODELS review
Folders checked	backend, frontend, MLP, LightGBM, CatBoost, FT-Transformer, Autoencoder, TASK_SVM
Documentation note	Keep extras/docs and Agents/knowledge files
Result	Project organized for model comparison and review

Jun 29, 2026

Backend/Frontend Integration and Final ALL_MODELS Review - Page 2 of 2

I also checked the documentation-related folders inside the project. Some documentation already exists under extras/docs and backend agent knowledge files, so I kept those files in place and treated them as part of the project structure.

The final point for this day was that the report should respect the existing documentation and only summarize the work in the main ML_Pipeline document. This avoids deleting or overwriting useful project notes.

My Understanding

My understanding is that this work is part of the transition from a single-pipeline report into a complete model-comparison workspace. The important point is to keep the same datasets, preprocessing logic, and output naming structure, so the final interpretation can focus on how the models behave differently.

Result / Next Step

Keep the model-specific folder structure clear and avoid mixing outputs between models.

- Keep the model-specific folder structure clear and avoid mixing outputs between models.

Use the same dataset names and model suffixes in result files.

- Use the same dataset names and model suffixes in result files.

Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

- Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

Respect the existing documentation already stored inside the project folders.

- Respect the existing documentation already stored inside the project folders.

Jun 30, 2026

Documentation Update and Final Report Organization - Page 1 of 2

I updated the main ML_Pipeline documentation so the work from June 18 to June 30 is written clearly day by day. The previous update was too compact, so I reorganized it into two pages per day as requested.

The purpose of this documentation update is to make the progress easier to follow. Instead of putting all the ALL_MODELS work into one summary, each day now has its own objective, completed work, outputs, and notes.

I also made sure to include the backend and frontend work because these parts are important for presenting and reviewing the model results.

Work Completed

Created a daily update from Jun 18 to Jun 30.

- Created a daily update from Jun 18 to Jun 30.

Used two pages for each day.

- Used two pages for each day.

Included model pipelines, MLflow, backend, frontend, and documentation notes.

- Included model pipelines, MLflow, backend, frontend, and documentation notes.

Kept the original report content before Jun 18 unchanged.

- Kept the original report content before Jun 18 unchanged.

Files / Areas Reviewed

Item	Details
Document update	Daily two-page section
Date range	Jun 18, 2026 to Jun 30, 2026
Main content	ALL_MODELS pipelines, MLflow, backend, frontend, documentation
Final status	Ready for review

Jun 30, 2026

Documentation Update and Final Report Organization - Page 2 of 2

The final document now gives a clearer timeline: first the baseline and XGBoost review, then the ALL_MODELS planning, then the new model pipelines, then all-model MLflow tracking, then backend/frontend dashboard work, and finally the documentation cleanup.

This version should be easier for Ashirbad, Karansinh, Dr. Vicky, or another reviewer to understand because the work is separated by date and each day has enough detail to explain what was done.

My Understanding

My understanding is that this work is part of the transition from a single-pipeline report into a complete model-comparison workspace. The important point is to keep the same datasets, preprocessing logic, and output naming structure, so the final interpretation can focus on how the models behave differently.

Result / Next Step

Keep the model-specific folder structure clear and avoid mixing outputs between models.

- Keep the model-specific folder structure clear and avoid mixing outputs between models.

Use the same dataset names and model suffixes in result files.

- Use the same dataset names and model suffixes in result files.

Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

- Continue comparing models using metrics, Top 3 files, SHAP outputs, and MLflow/dashboard views.

Respect the existing documentation already stored inside the project folders.

- Respect the existing documentation already stored inside the project folders.